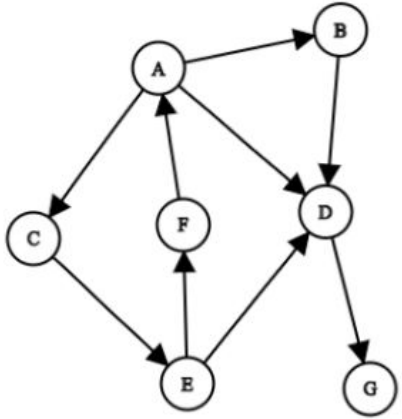
Teoría de Grafos

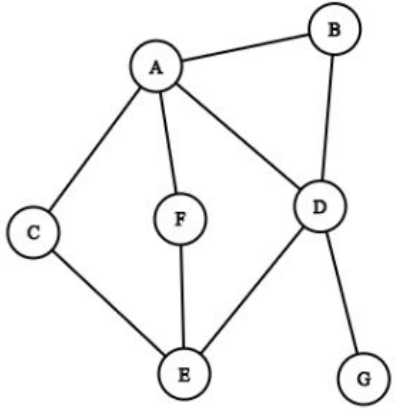
ICPC Campamento de Programación Oruro 2024

Definición

Un grafo es un conjunto de objetos llamados vértices o nodos, unidos por enlaces llamados aristas o arcos.



Grafo dirigido



Grafo no dirigido

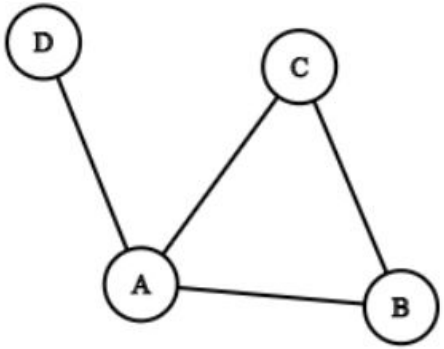
Ejemplos

- En Facebook, dos usuarios, pueden o no ser amigos. Esta relación es mutua, es decir que puede pensarse como un grafo no dirigido, en donde cada usuario es un nodo y cada relación de amistad es una arista.
- En Instagram, un usuario puede seguir a otro, sin necesidad que ese otro usuario lo siga a uno. Es decir que puede pensarse como un grafo dirigido, en donde cada usuario es un nodo y cada relación seguidor-seguido es una arista.

En general, dada cualquier situación en la que tenemos pares de cosas relacionadas, probablemente sirve verla como un grafo.

Matriz de adyacencia

La matriz de adyacencia es una matriz de n × n donde n es la cantidad de nodos del grafo, que en la posición (i, j) tiene un 1 (o true) si hay una arista entre los nodos i y j, o 0 (o false) en caso contrario. Uso de memoria: O(n^2)



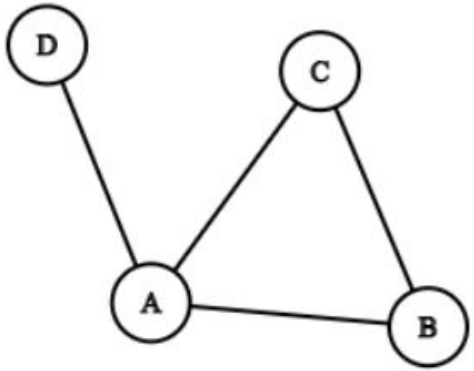
$$\begin{array}{c c} & \begin{array}{cccc} A & B & C & D \end{array} \\ \begin{array}{c} A \\ B \\ C \\ D \end{array} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

Matriz de adyacencia

```
int n,m;
cin>>n>>m;
vector<vector<int>> g(n, vector<int>(n));
for(int i=0;i<m;i++){
    int a,b;
    cin>>a>>b;
    g[a][b] = 1;
    g[b][a] = 1;
}
```

Listas de adyacencia

La lista de adyacencia es un vector de vectores de enteros, que en el i-ésimo vector tiene el número j si hay una arista entre los nodos i y j. Uso de memoria: O(m), donde m es la cantidad de aristas.



A :	{B,C,D}
B :	{A,C}
C :	{A,B}
D :	{A}

Listas de adyacencia

```
vector<vector<int>> g;
void lista_adyacencia()
{
    int n,m;
    cin>>n>>m;
    g.resize(n);
    for(int i=0;i<m;i++){
        int a,b;
        cin>>a>>b;
        g[a].pb(b);
        g[b].pb(a);
    }
}
```

Algunas definiciones

Vecino y grado: Dada una arista que conecta dos vértices u y v, decimos que u es vecino de v (y que v es vecino de u). Además, a la cantidad de vecinos de u se le llama el grado de u.

Distancia: Definimos la distancia de u a v como el menor n tal que hay un camino de largo n de u a v.

BFS

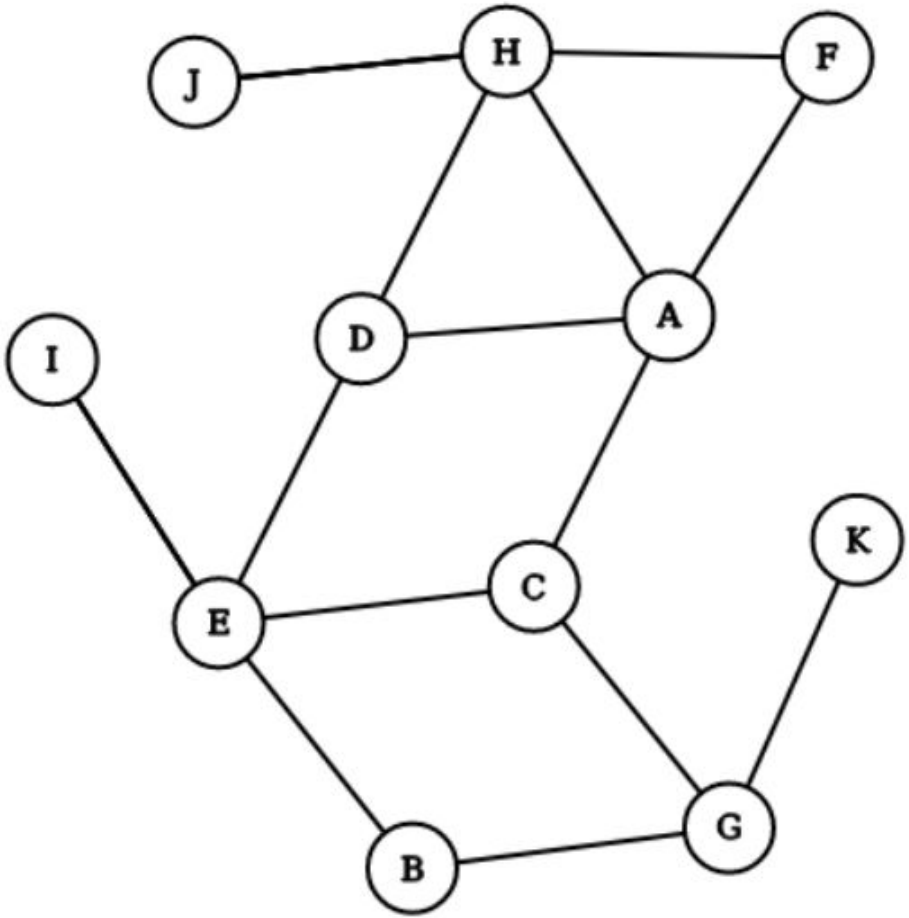
BFS (siglas de breadth-first-search o “búsqueda por amplitud”) es un algoritmo para recorrer grafos, que a su vez sirve para calcular la distancia mínima desde un nodo v a cada uno de los otros.

- Inicialmente se procesa v, que tiene distancia a sí mismo 0, por lo que seteamos d[v] = 0. Además es el único a distancia 0.
- Los vecinos de v tienen sí o sí distancia 1, por lo que seteamos d[y] = 1 para todo y vecino de v. Además, estos son los únicos a distancia 1.

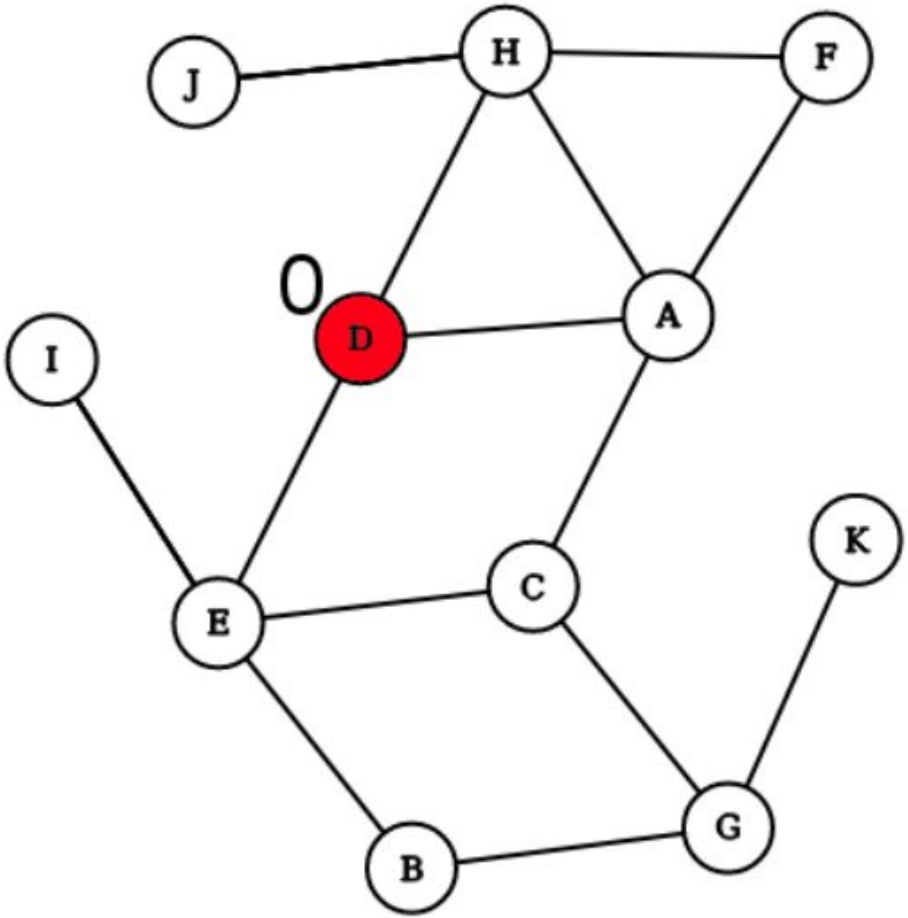
BFS

- Ahora tomamos los nodos a distancia 1, y para cada uno nos fijamos en sus vecinos cuya distancia aún no calculamos. Estos son los nodos a distancia 2.
- Así sucesivamente, para saber cuáles son los nodos a distancia k + 1, tomamos los vecinos de los nodos a distancia k que no hayan sido visitados aún.

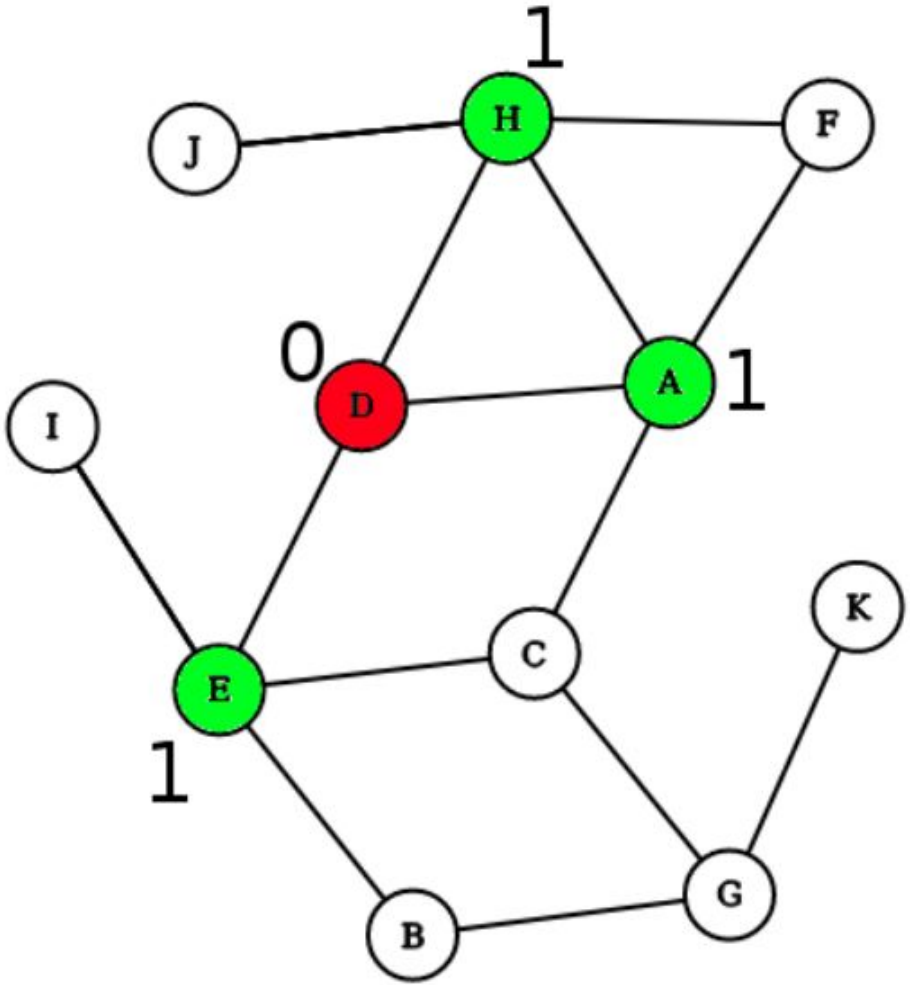
BFS



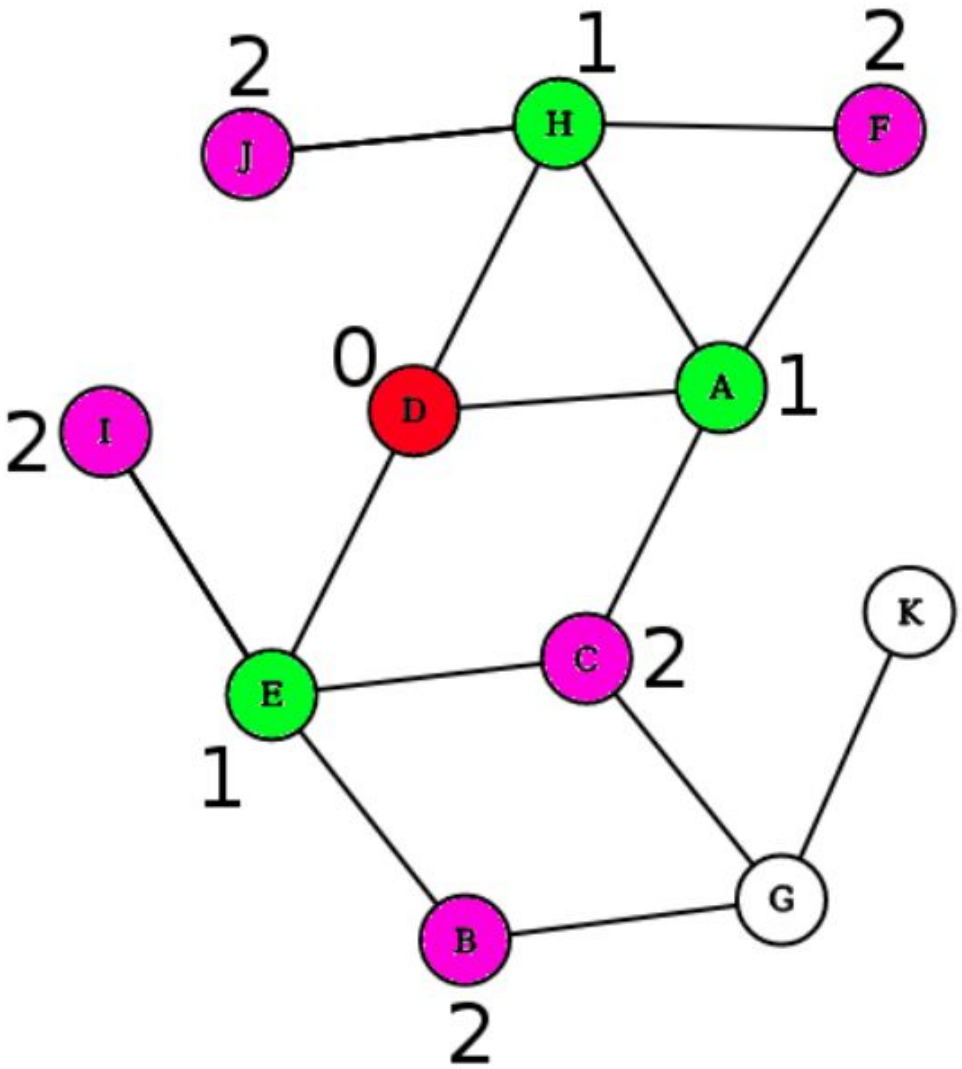
BFS



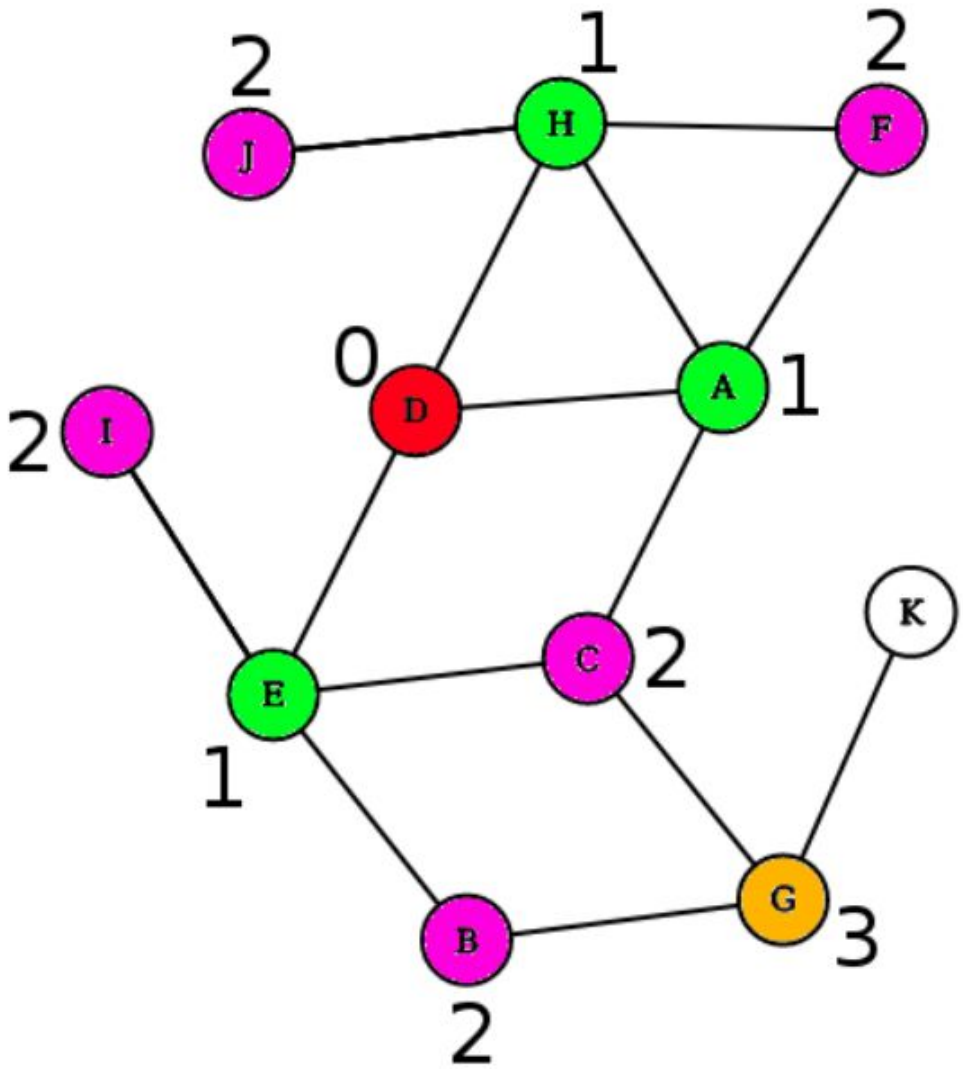
BFS



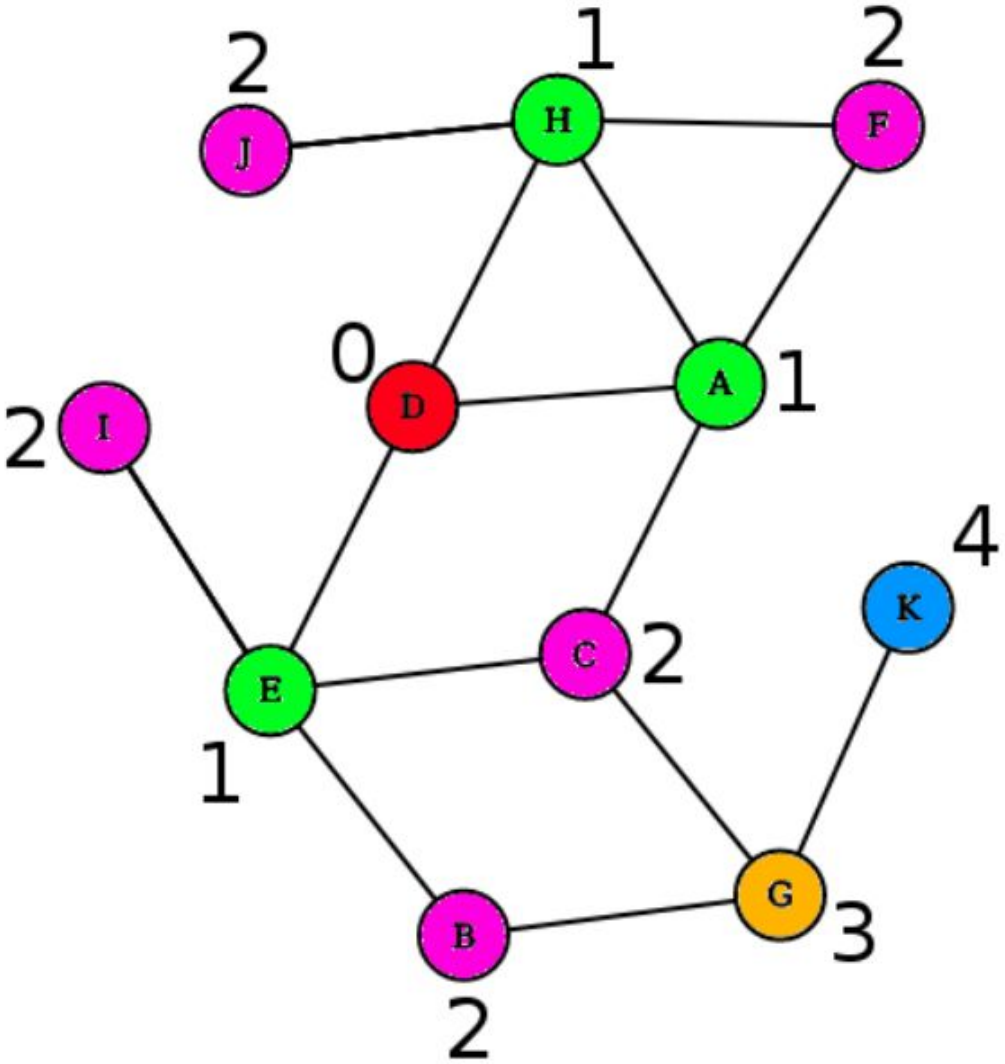
BFS



BFS



BFS



BFS CÓDIGO

```
vector<int> bfs(vector<vector<int>>& g , int v) {
    vector<int> dis(g.size(), -1); // vector de distancias
    queue<int> q;
    dis[v] = 0;
    q.push(v);
    while(!q.empty()) { // mientras haya nodos por procesar
        int node= q.front();
        q.pop();
        for(int x : g[node]) { // para cada y vecino de x
            if(dis[x] == -1) { // si x no fue encolado aun
                dis[x] = dis[node] + 1;
                q.push(x);
            }
        }
    }
    return dis;
}
```

BFS complejidad

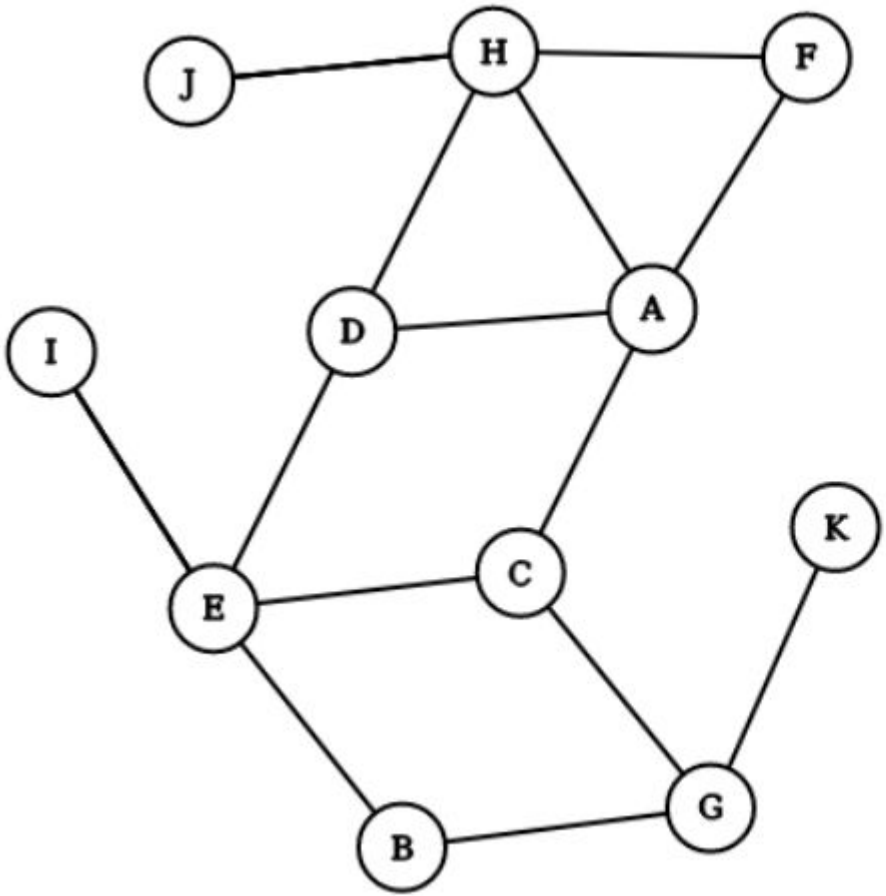
- Cada nodo lo procesamos una sóla vez, porque cuando lo agregamos a la cola, también inicializamos su distancia, por lo que no volverá a ser agregado.
- Como cada nodo es procesado una sóla vez, entonces cada arista es procesada una sóla vez (o una vez en cada sentido para no-dirigidos).
- Entonces, la complejidad de BFS es O(m), donde m es la cantidad de aristas.

DFS

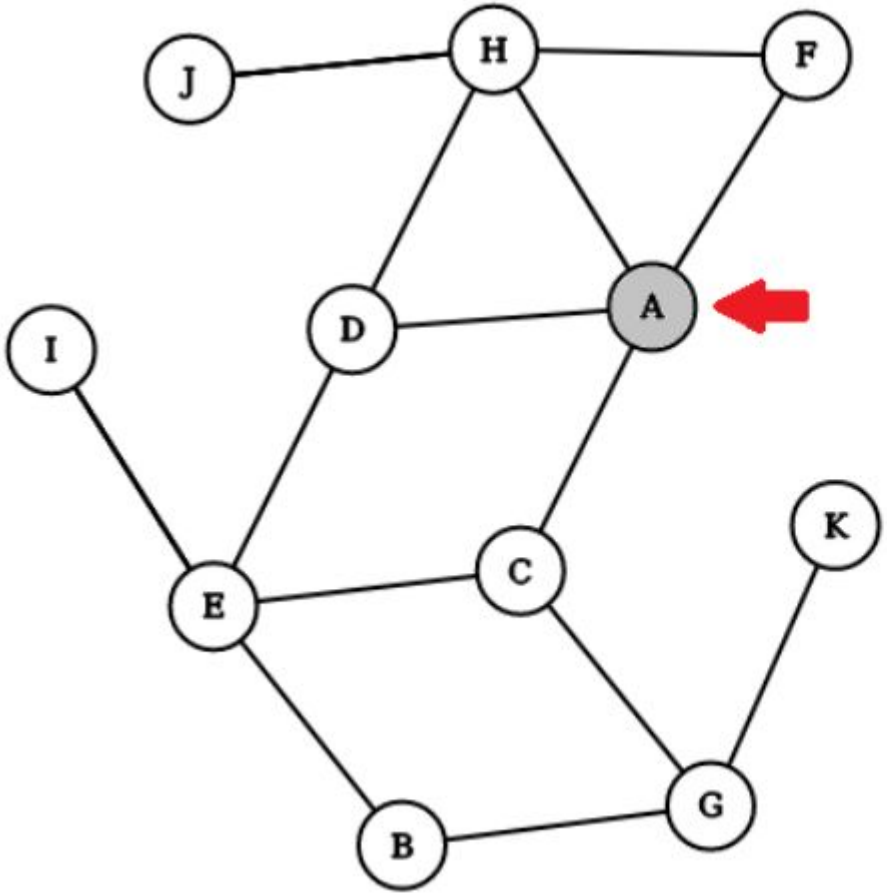
Otra manera de recorrer un grafo es con DFS (depth-first-search o “búsqueda por profundidad”):

- Arrancamos de cierto nodo y lo marcamos como visitado.
- Luego, evaluamos sus vecinos de a uno, y cada vez que encontramos uno que no esté marcado como visitado, seguimos procesando a partir de él. Este procedimiento es recursivo, por lo que se puede implementar así.
- Cuando no quedan vecinos por visitar salgo para atrás (vuelvo en la recursión).

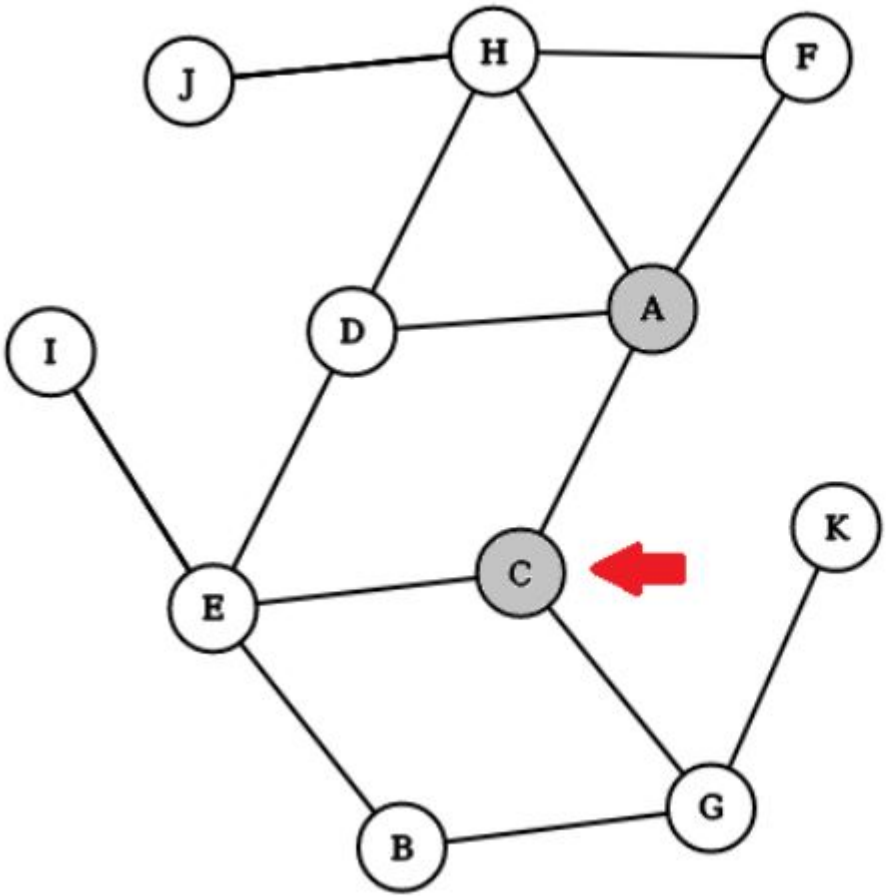
DFS



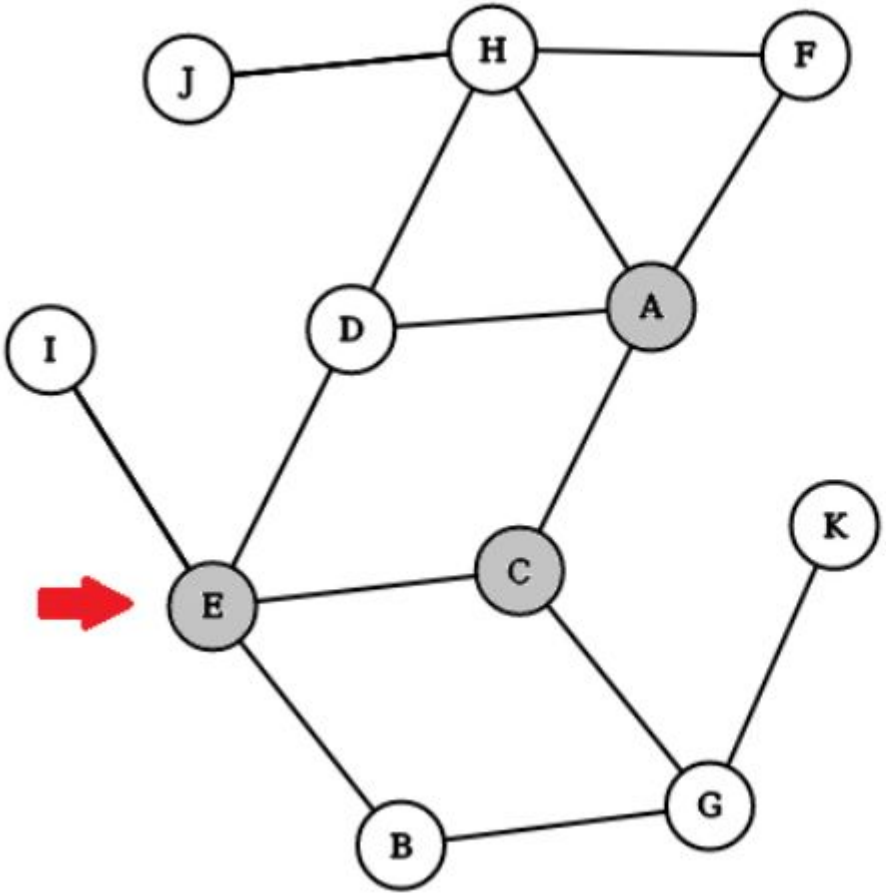
DFS



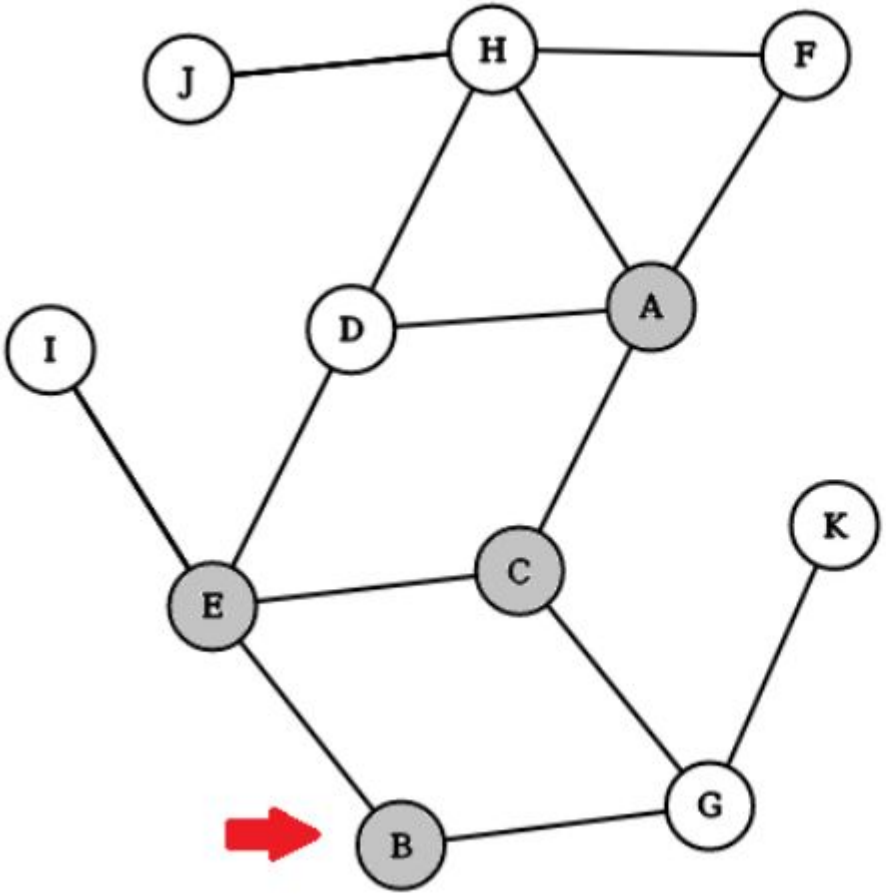
DFS



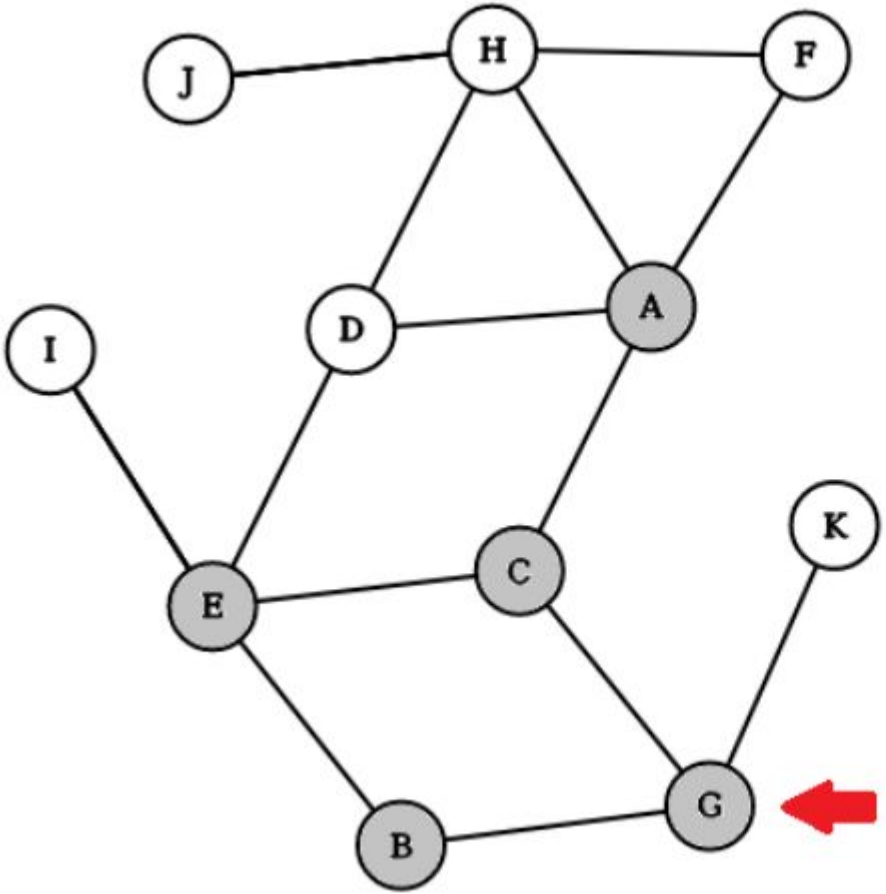
DFS



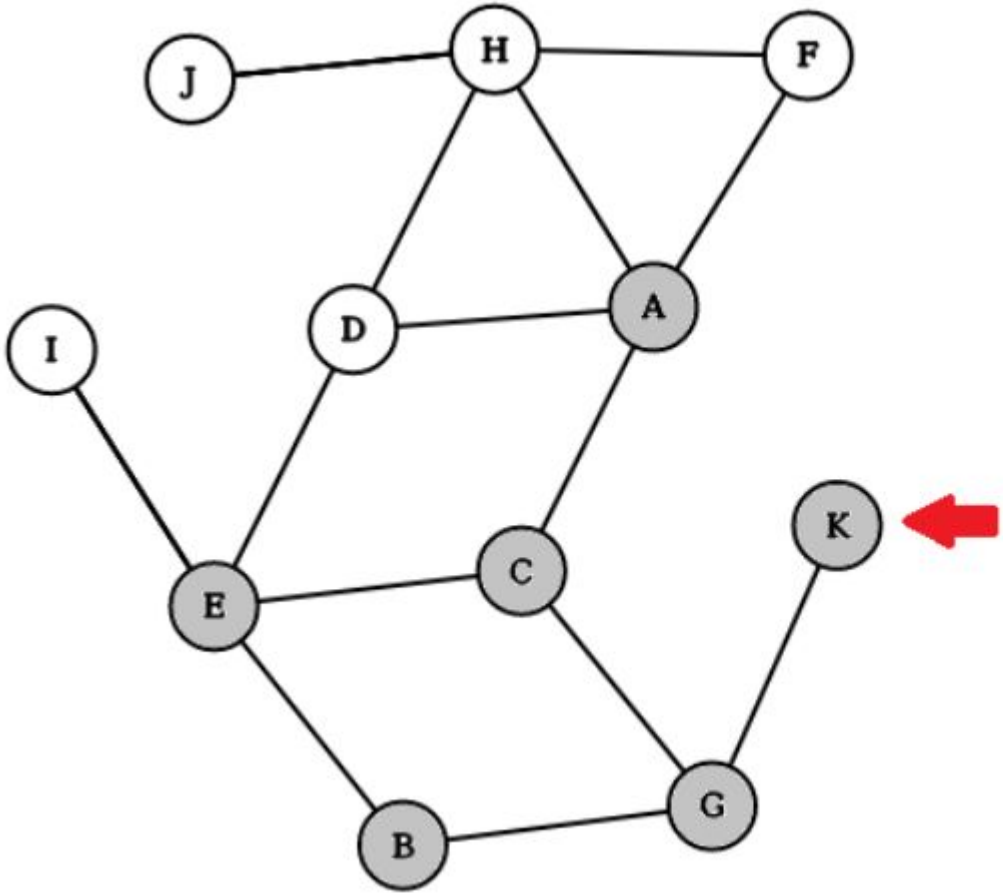
DFS



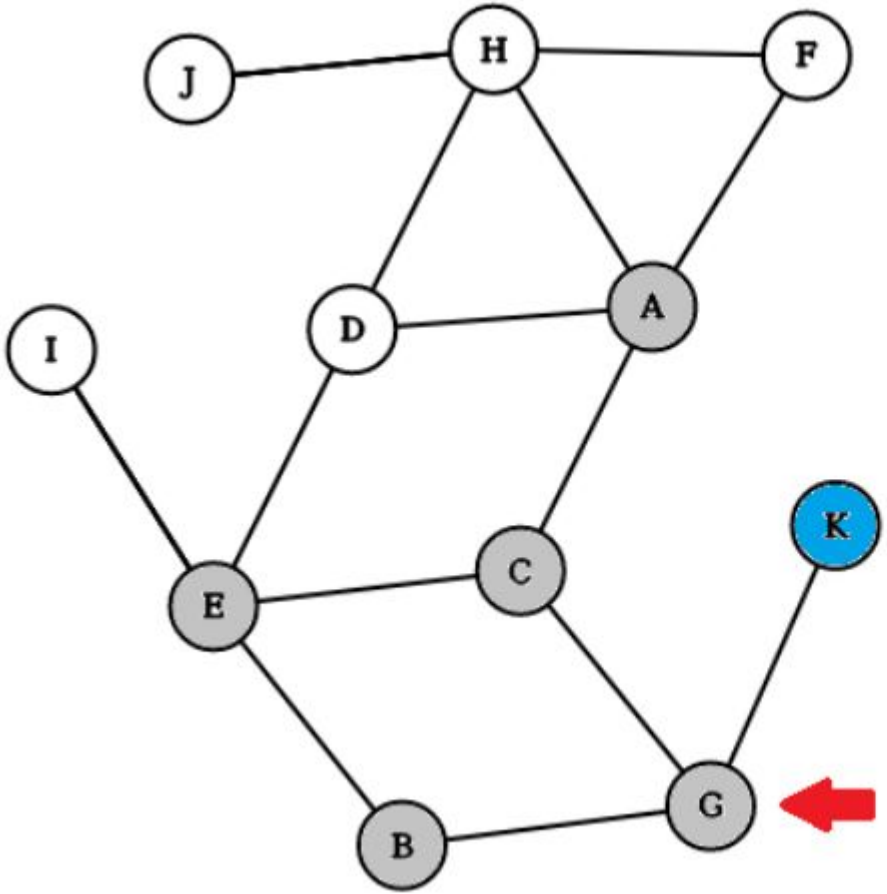
DFS



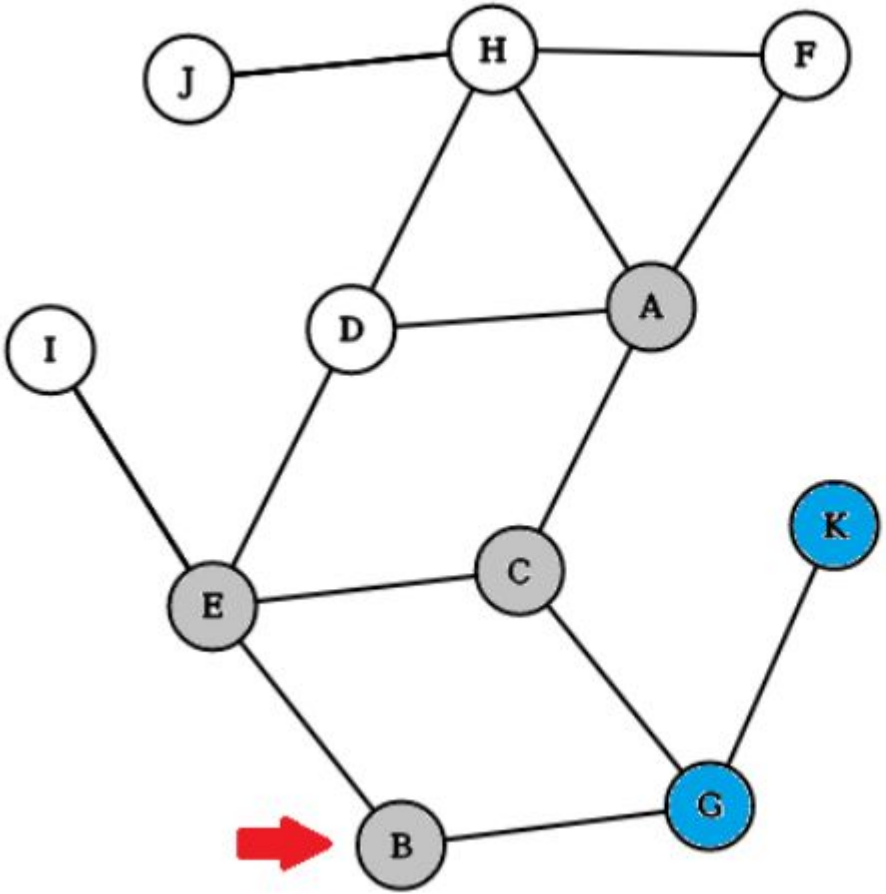
DFS



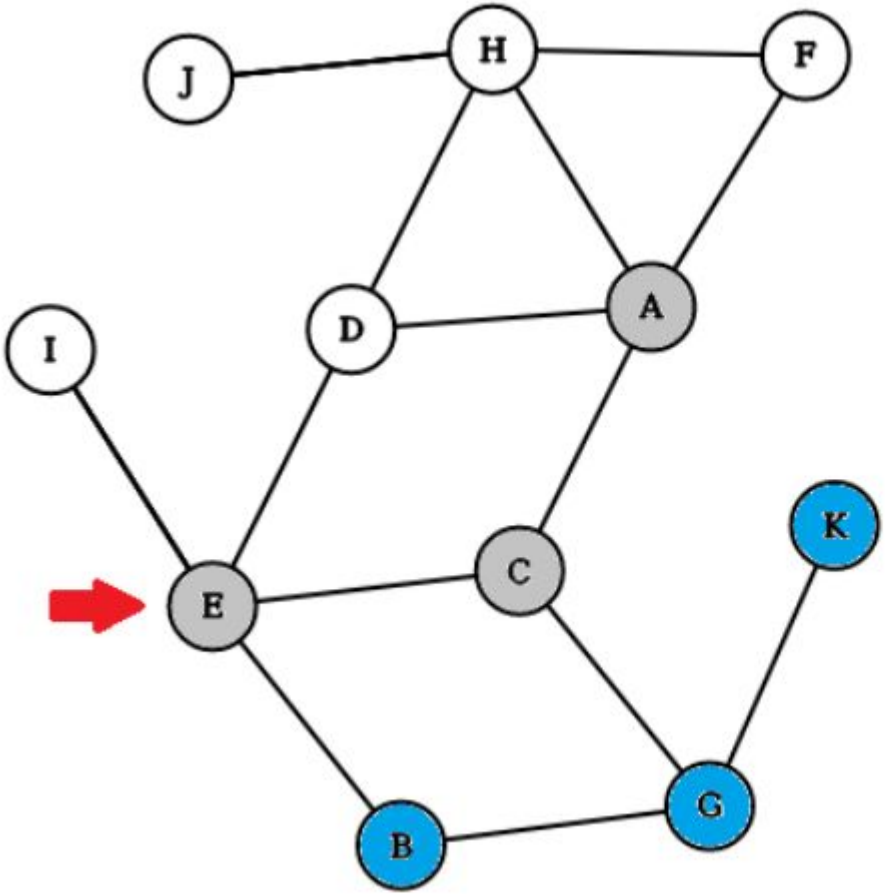
DFS



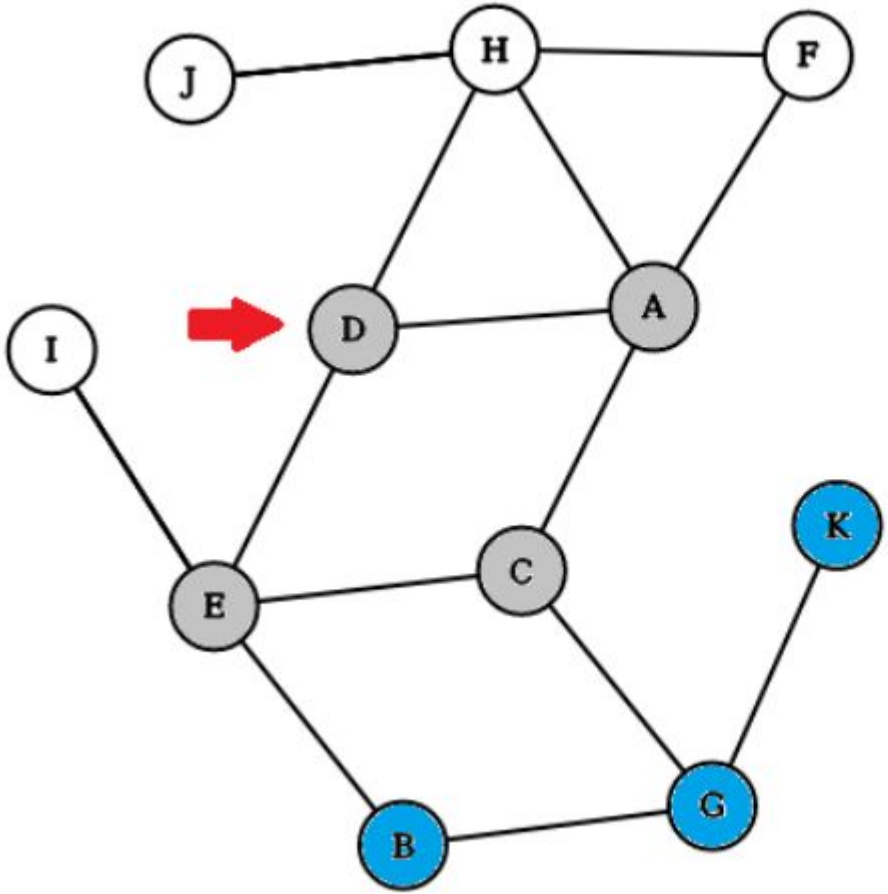
DFS



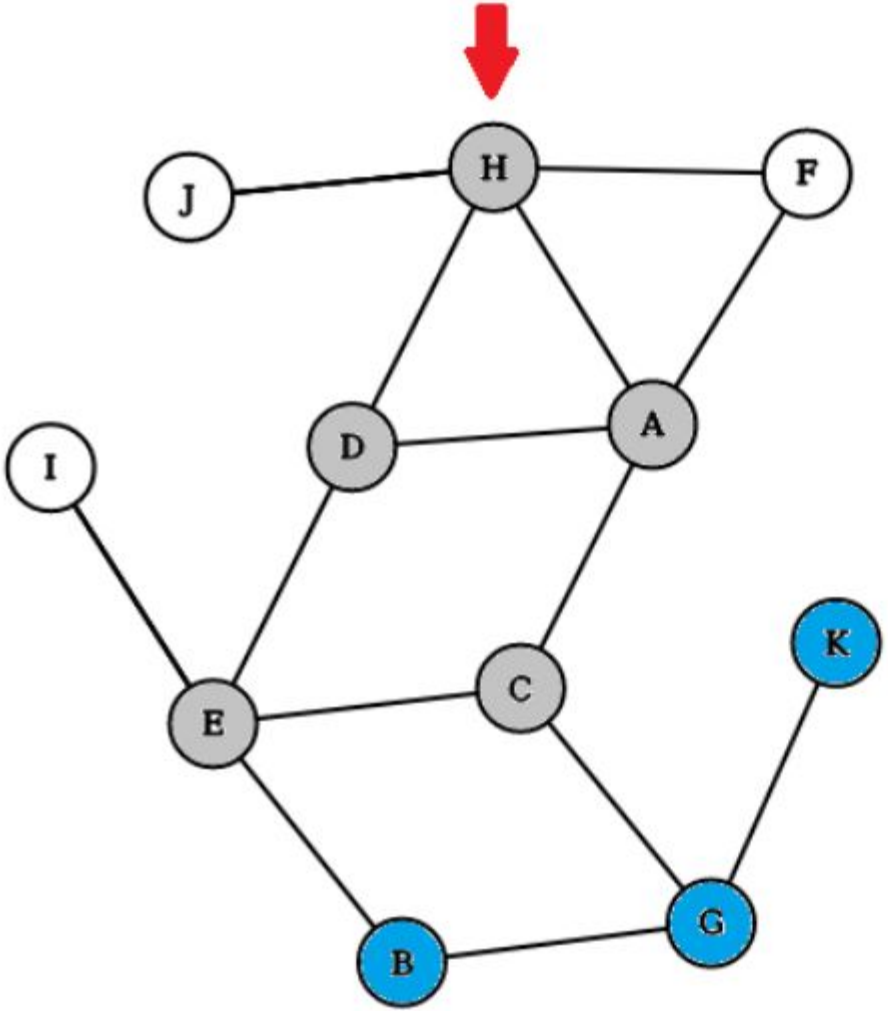
DFS



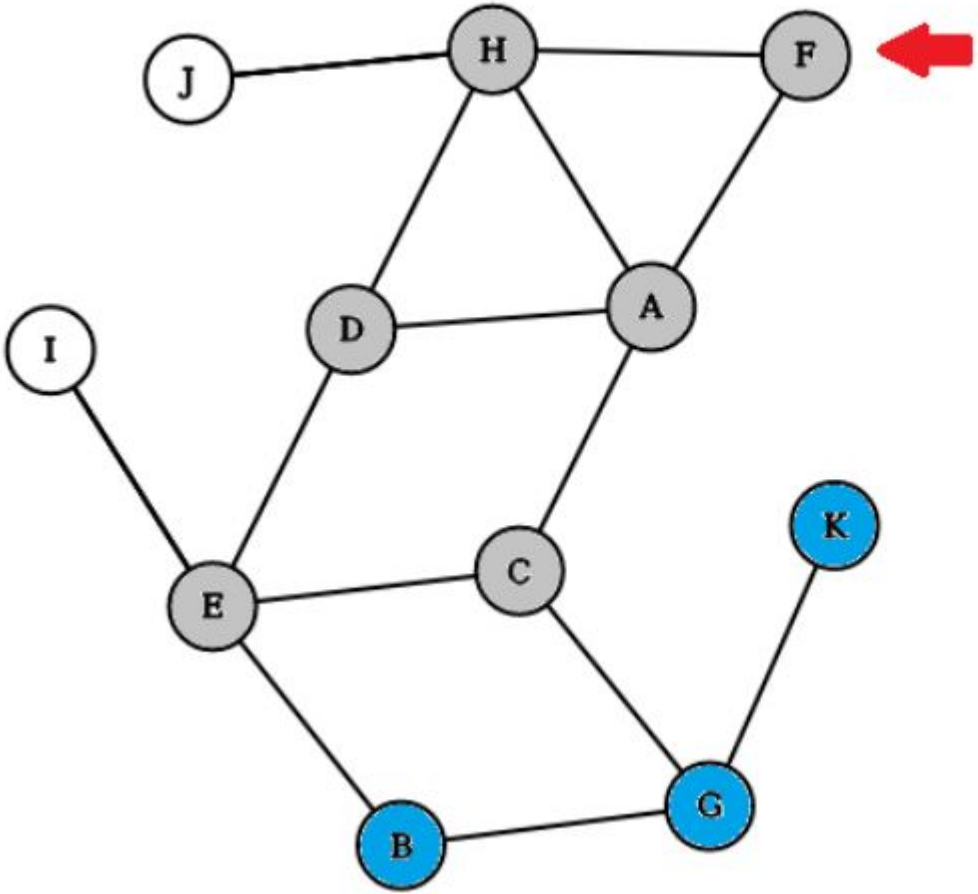
DFS



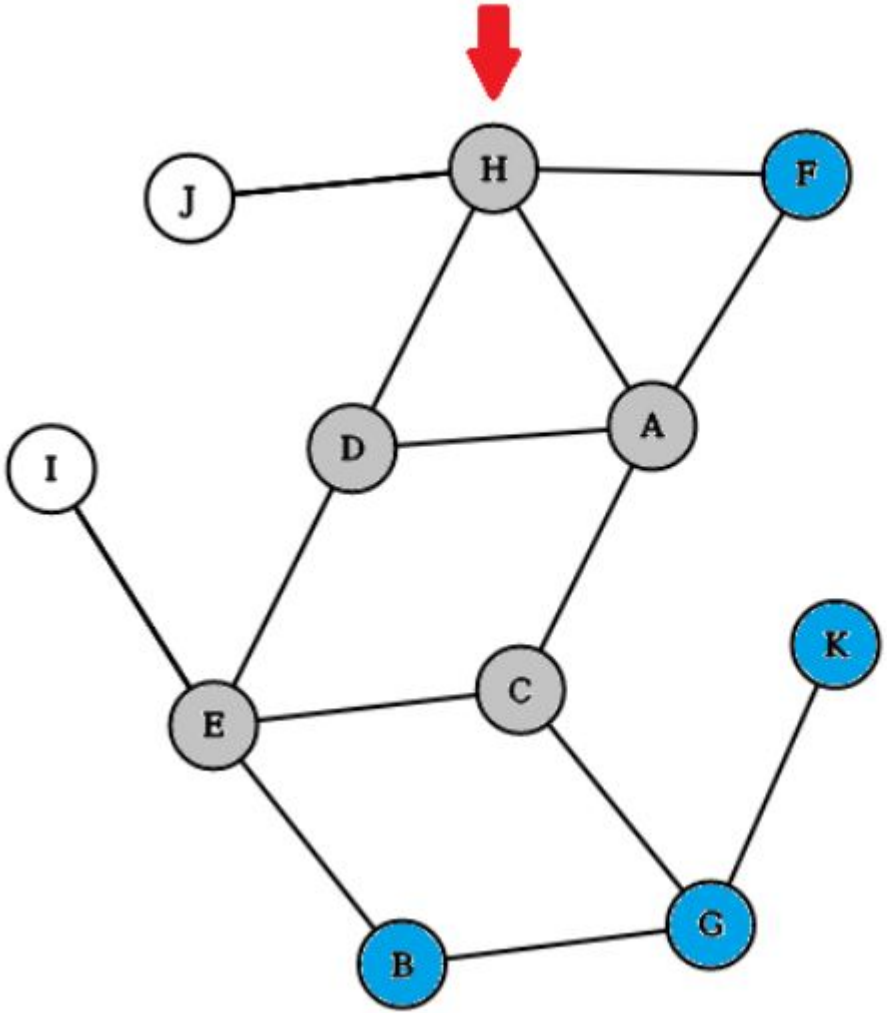
DFS



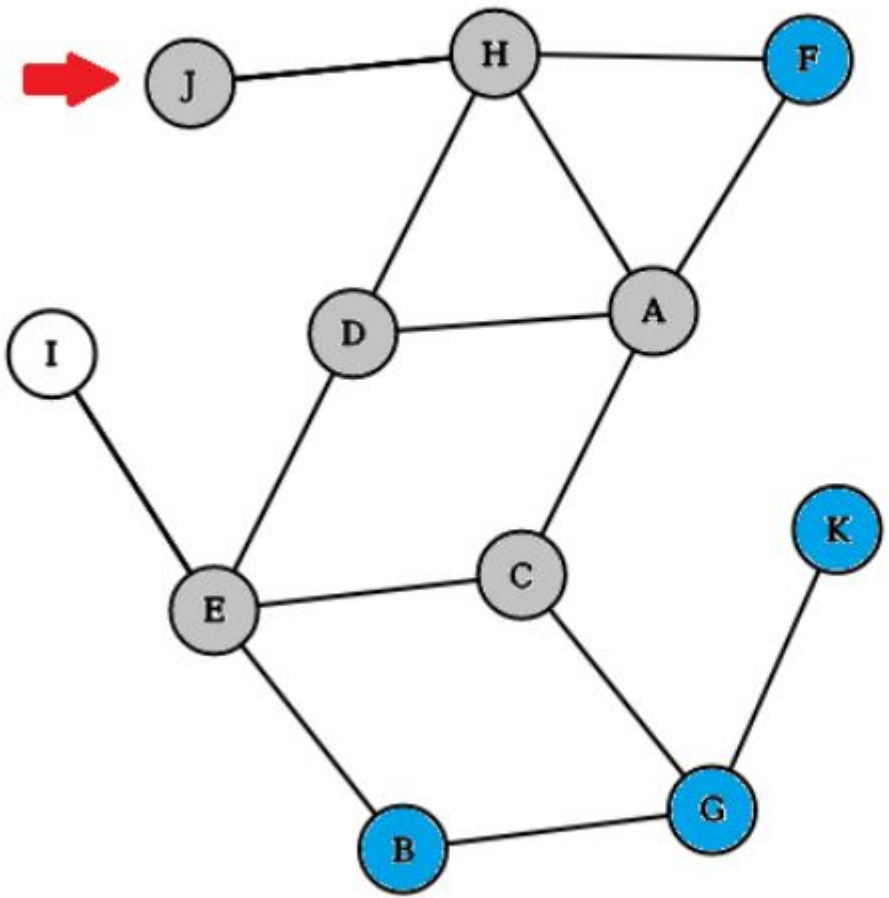
DFS



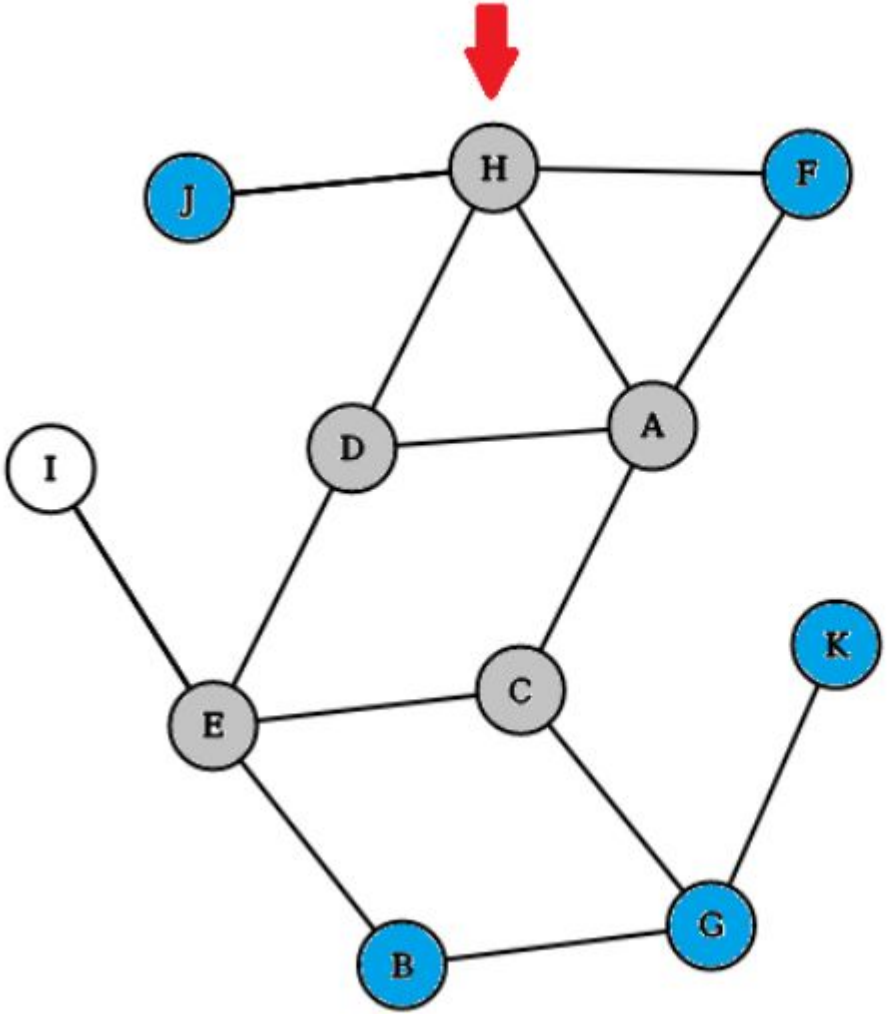
DFS



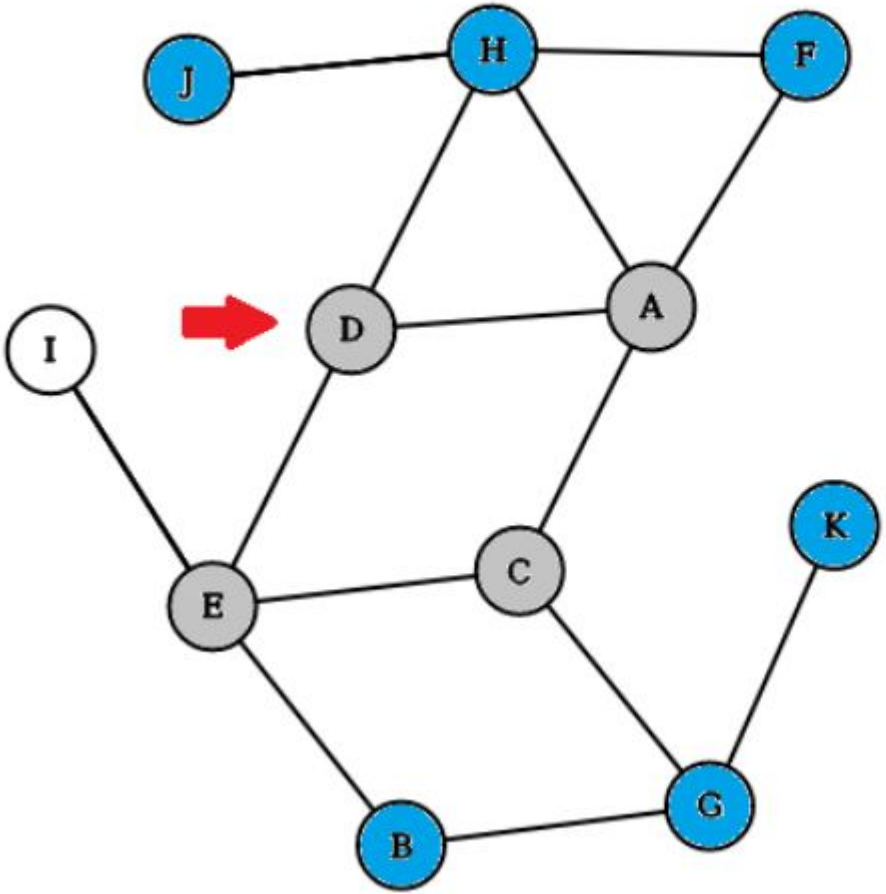
DFS



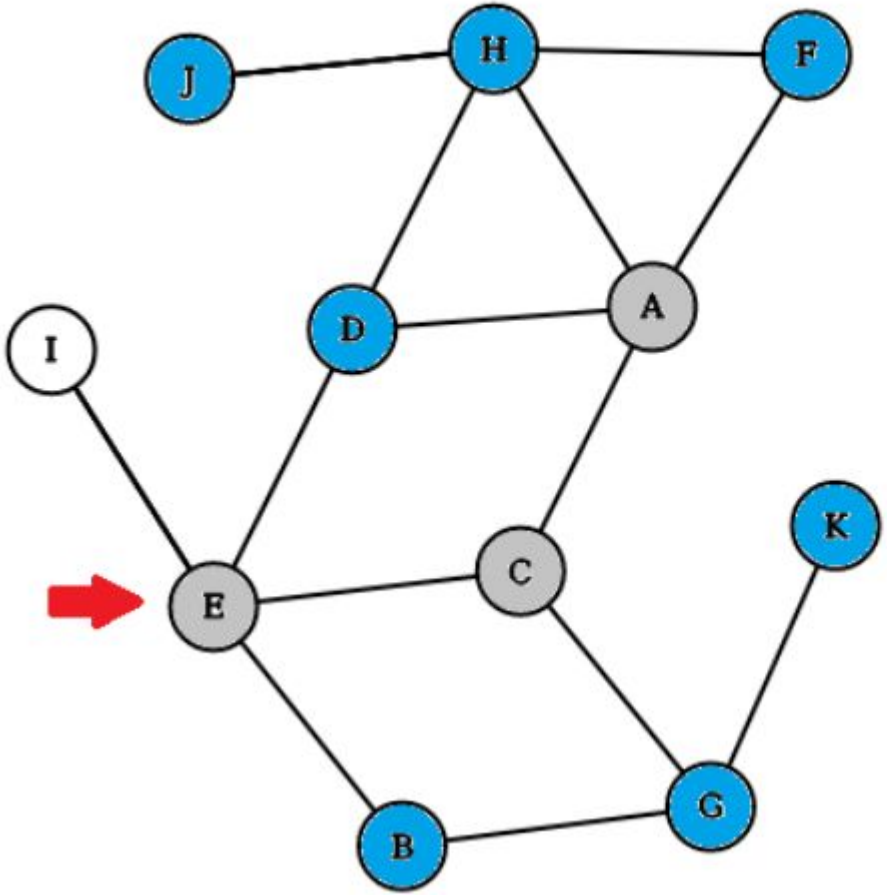
DFS



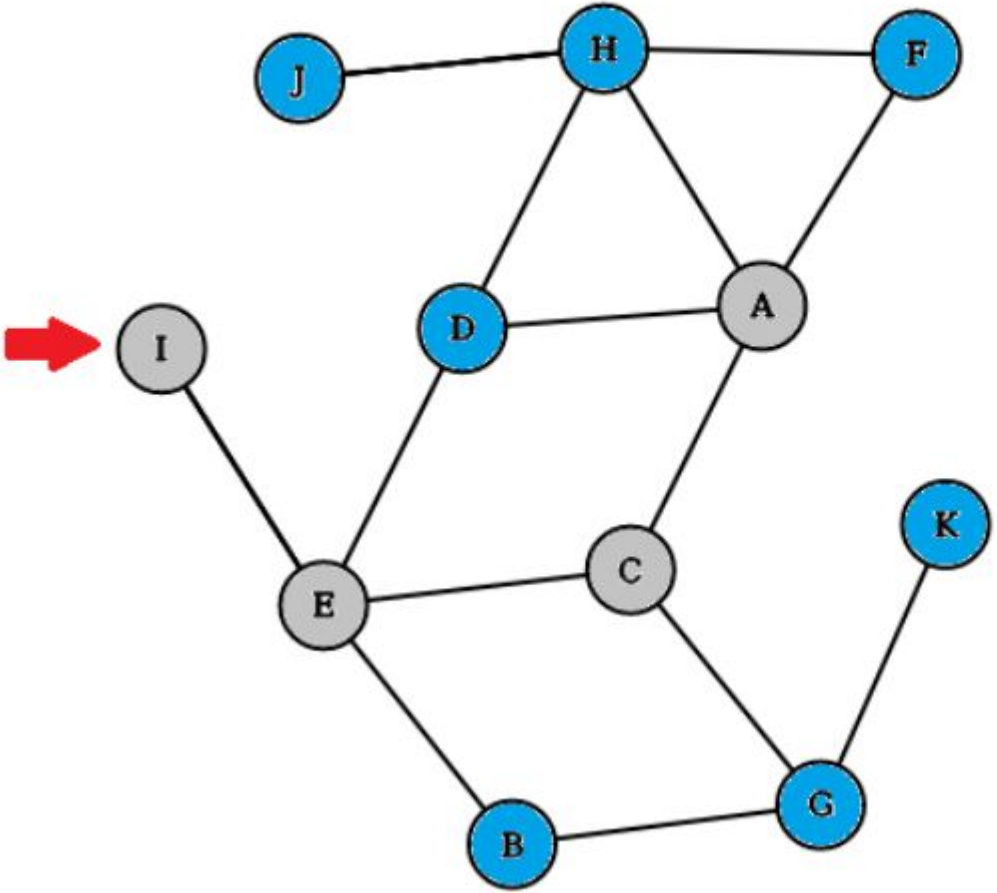
DFS



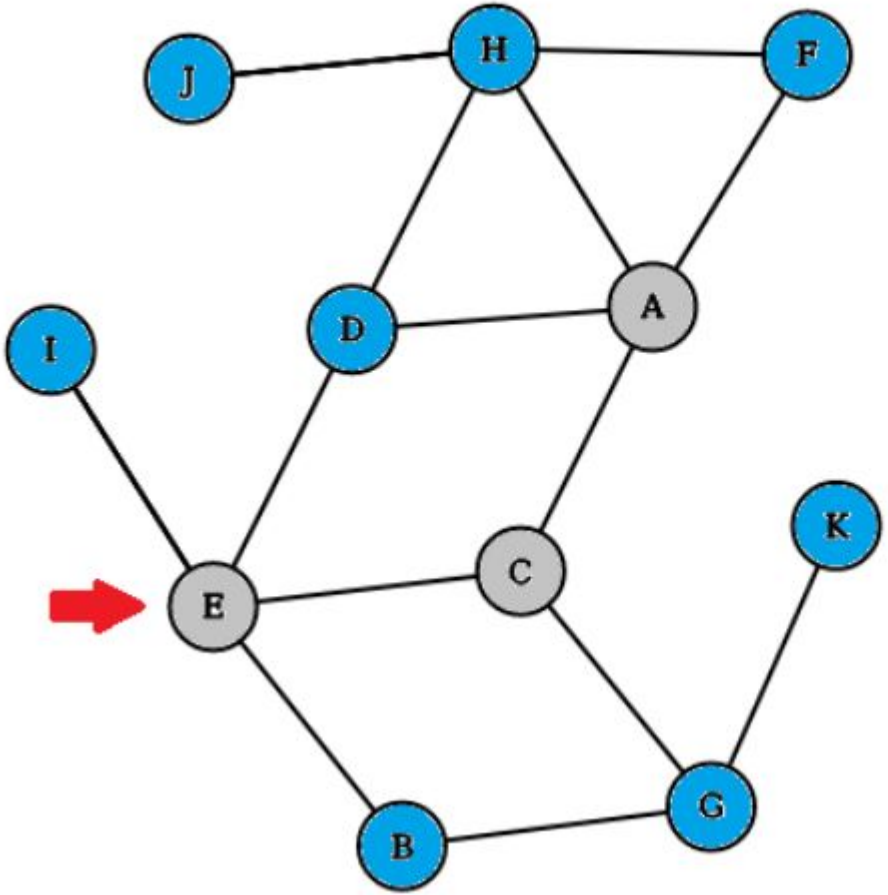
DFS



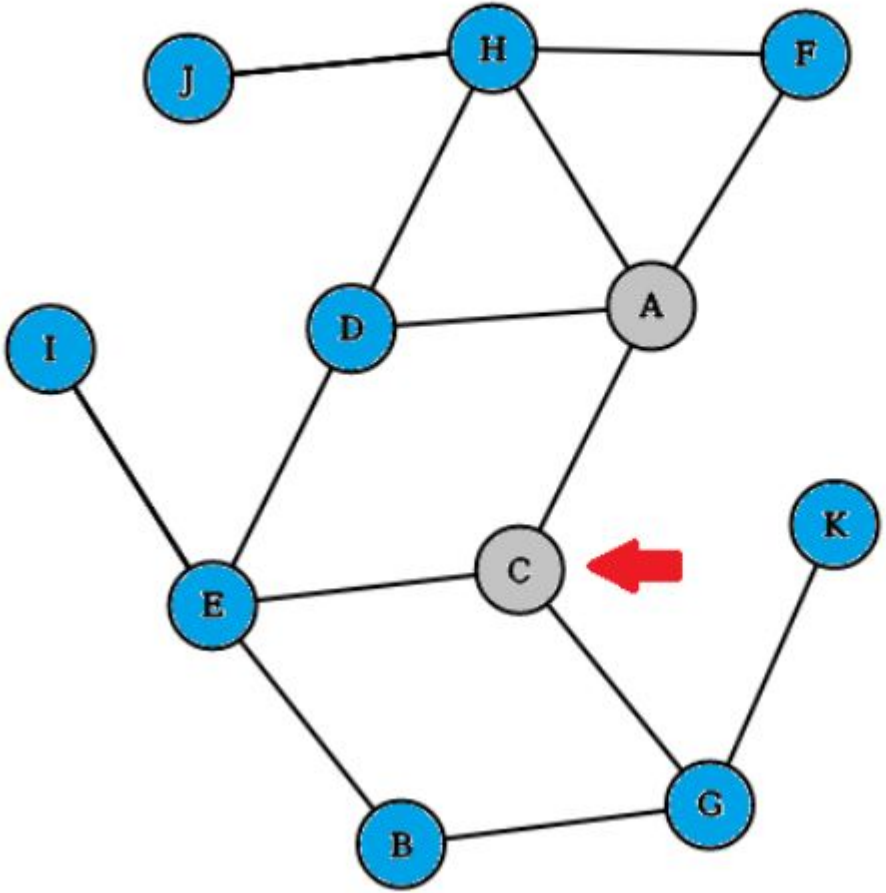
DFS



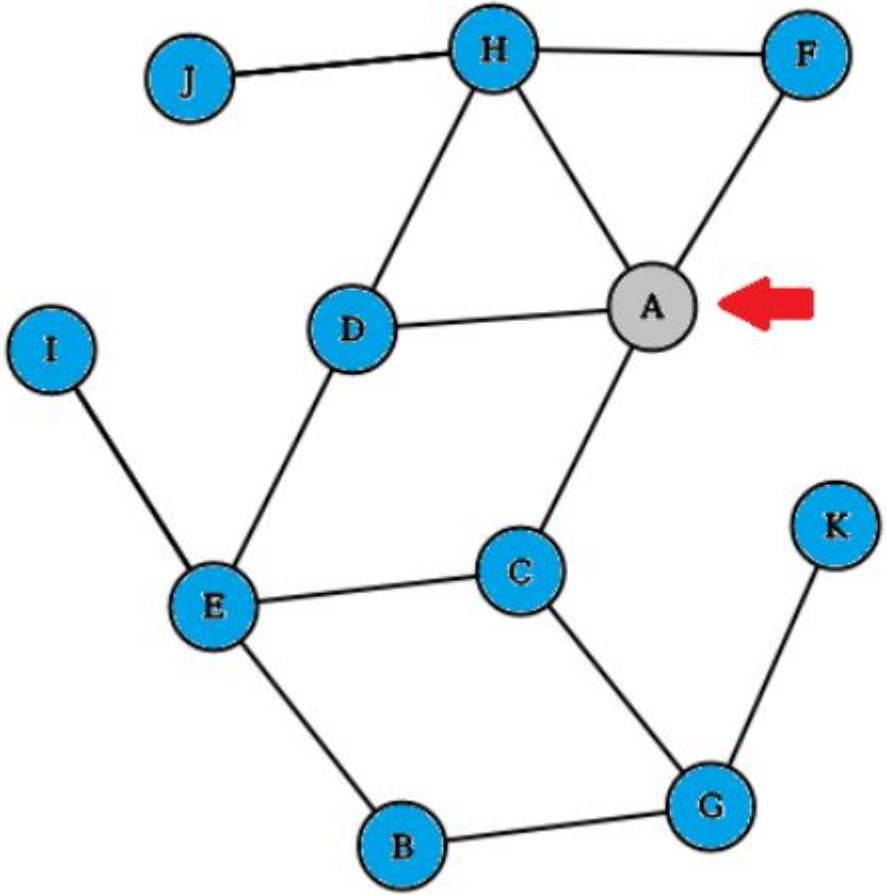
DFS



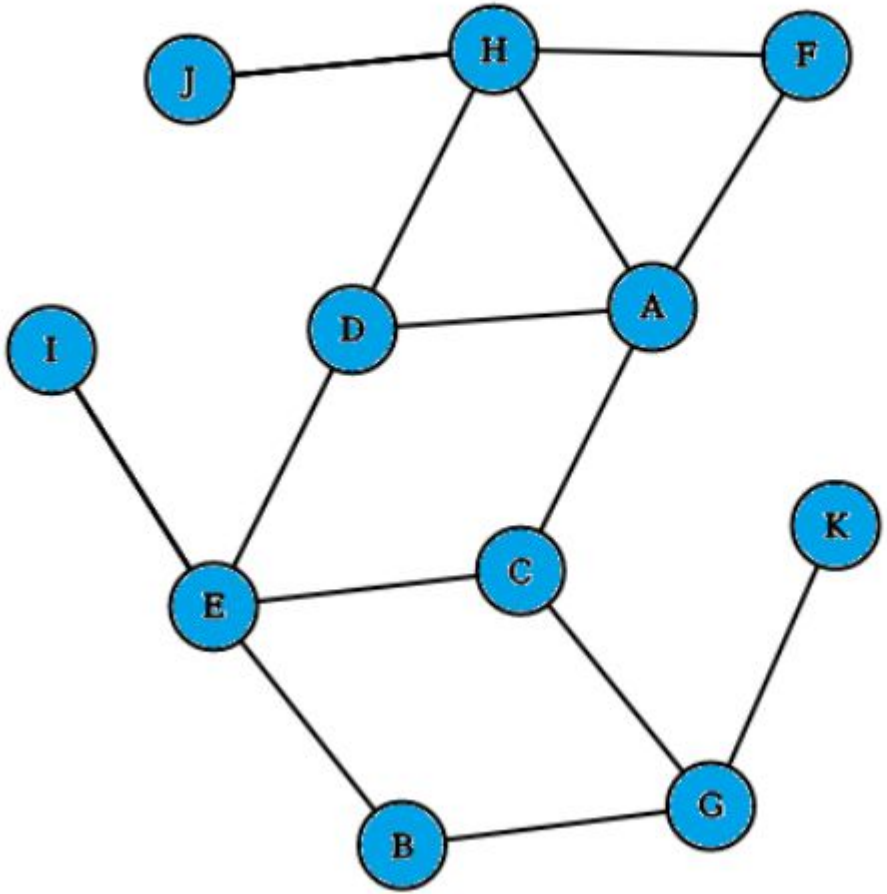
DFS



DFS



DFS



DFS Implementación

```
vector<bool> vis(tam);
void dfs(int node)
{
    vis[node] = 1;
    for(int x : g[node])
        if(!vis[x])
            dfs(x);
}
```

DFS

DFS también es O(m). BFS y DFS son similares: ambos procesan todos los nodos alcanzables desde cierto nodo, pero lo hacen en distinto orden. Muchos problemas salen con ambas técnicas, por lo que podemos usar la que nos quede más cómodo.

Algunos problemas sólo se resuelven con una de ellas:

BFS: distancias mínimas.

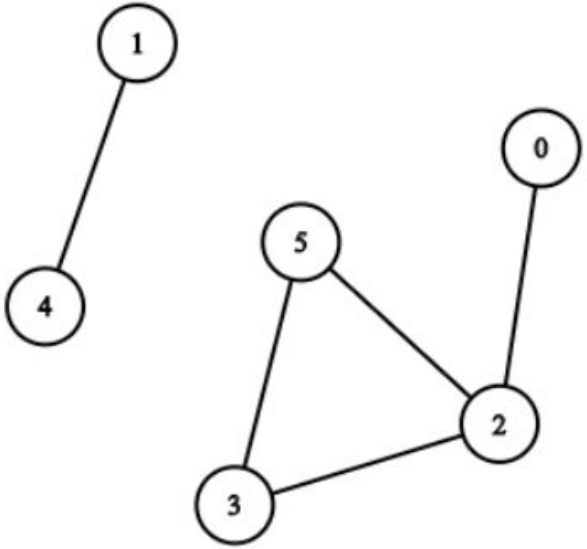
DFS: algunos problemas que tienen que ver con la estructura del grafo, como por ejemplo encontrar puentes y puntos de articulación.

Componentes Conexas

Decimos que dos vértices u y v están conectados si hay un camino (secuencia de aristas con vértice en común) que los une.

- Para todo grafo no-dirigido, podemos particionar el conjunto de nodos en varios subconjuntos, tales que dos nodos están conectados si y sólo si pertenecen al mismo subconjunto.
- Estos subconjuntos se denominan componentes conexas del grafo.

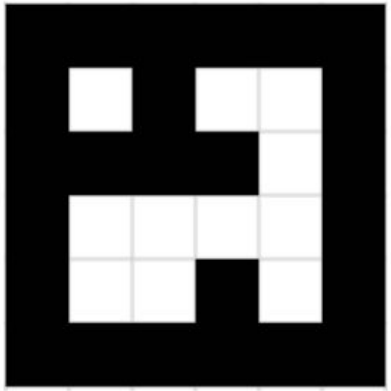
Componentes Conexas

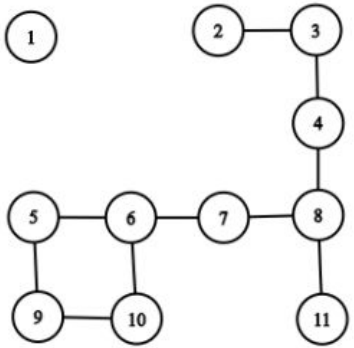


Las componentes conexas son $\{1, 4\}$ y $\{0, 2, 3, 5\}$

Flood Fill

Flood Fill es un algoritmo utilizado para determinar y llenar una área conectada en una matriz bidimensional.





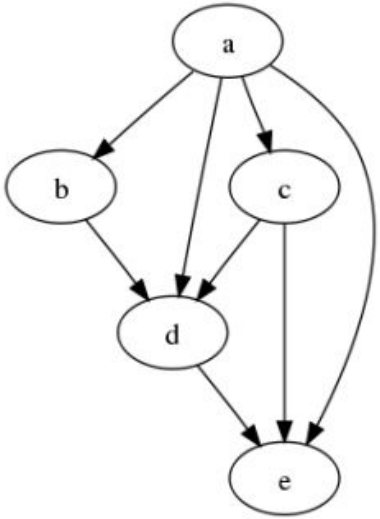
Un tablero bidimensional puede ser tratado como un grafo implícito donde cada celda es un nodo y las adyacencias (celdas arriba, abajo, izquierda y derecha) representan aristas.

Flood fill

```
int n, m; // dimensiones del tablero
int dir[2][4] = {{0,0,1,-1}, {1,-1,0,0}}; // direcciones
int[MAXN][MAXM] tab, visi;
// en el tablero los 1's representan obstáculos
void floodfill(int x, int y) {
    if(x < 0 || y < 0 || x >= n || y >= m || visi[x][y] || tab[x][y] == 1)
        return;
    visi[x][y] = 1;
    for(int i = 0; i < 4; i++)
        floodfill(x + dir[0][i], y + dir[1][i]);
}
```

Orden Topológico

Dado un grafo dirigido, un orden topológico es un ordenamiento de los nodos de modo que para cada arista x→y, x viene antes que y en el orden.



Este grafo tiene dos órdenes topológicos:

- a,b,c,d,e
- a,c,b,d,e

Orden Topológico

Un grafo tiene orden topológico si y sólo si no tiene. A estos grafos se los llama DAG (directed-acyclic-graph o “grafo dirigido sin ciclos”).

Para obtener el orden topológico en un DAG:

- Obtenemos el grado de entrada de cada nodo y metemos los de grado 0 en una cola.
- Mientras que la cola no esté vacía, tomamos el nodo del frente y “lo eliminamos” del grafo, actualizando el grado de entrada de sus vecinos y metiendo a la cola los que pasen a tener grado 0.

Orden Topológico

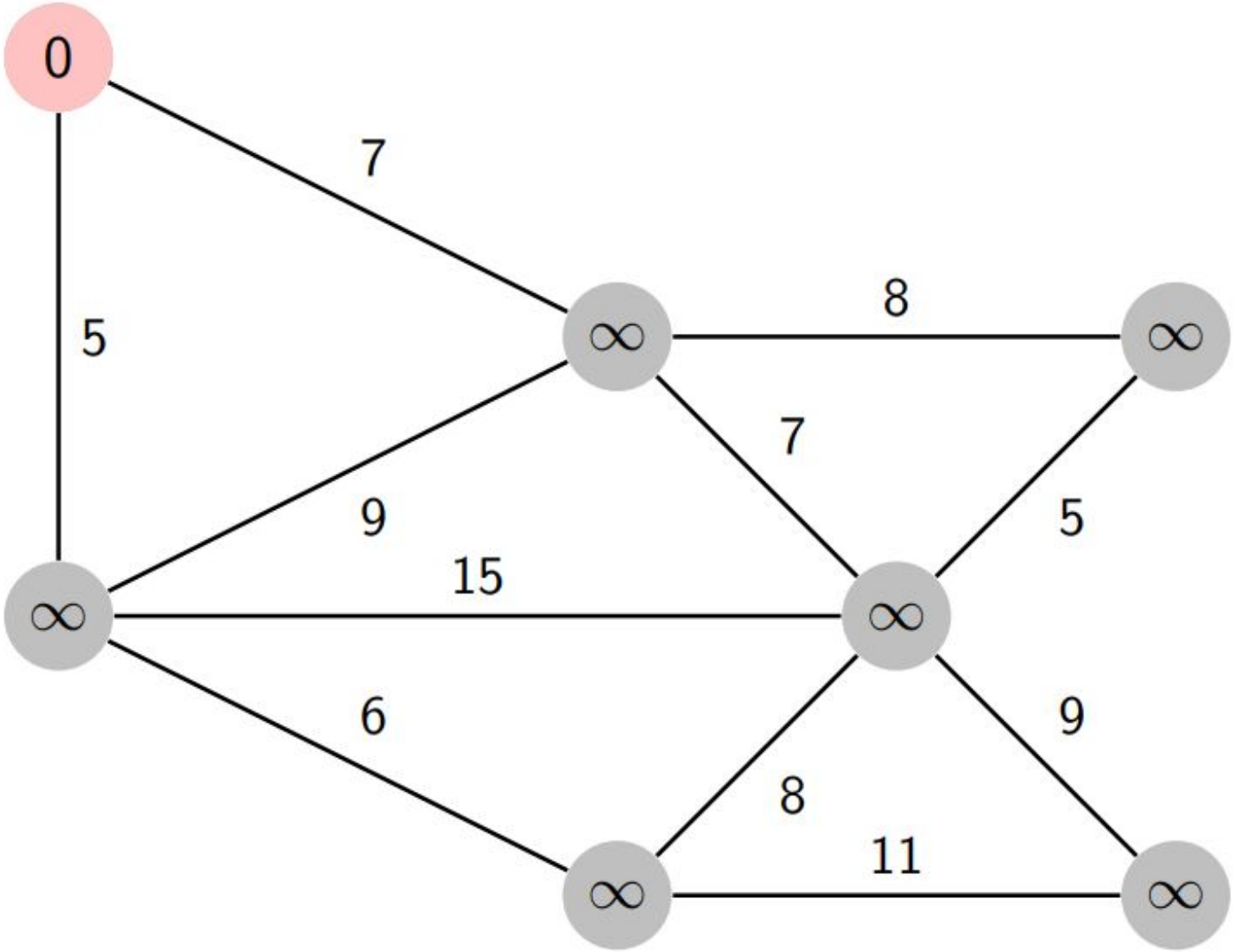
```
vector<int> topSort(vector<vector<int>> &g, vector<int> in)
{
    int n = in.size();
    vector<int> topo;
    queue<int> q;
    for(int i = 0; i < n; i++)
        if(in[i] == 0)
            q.push(i);
    while(!q.empty()) {
        int node = q.front(); q.pop();
        topo.push_back(node);
        for(int x : g[node])
            if(--in[x] == 0)
                q.push(x);
    }
    return topo;
}
```

Dijkstra

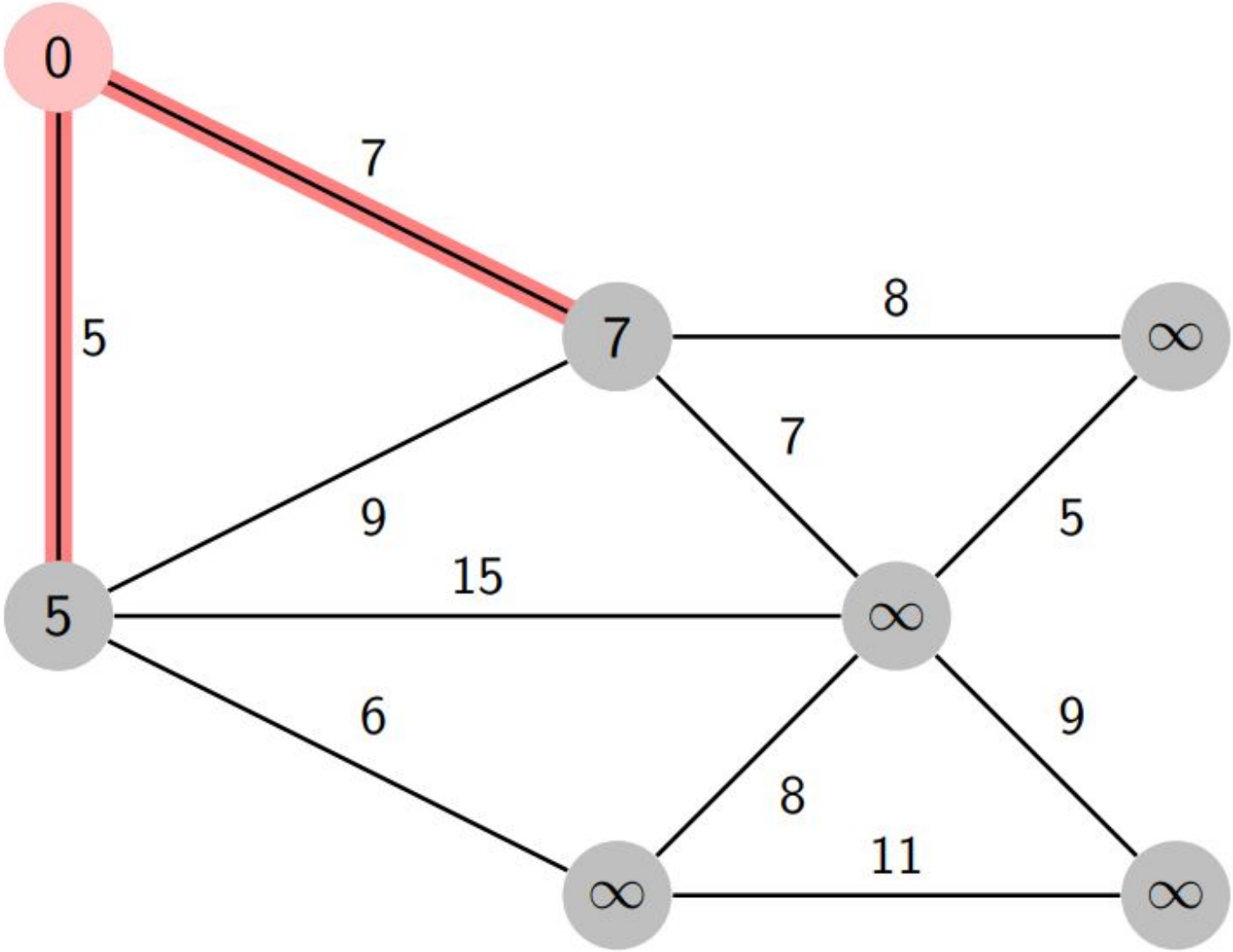
Dado un nodo (v), el algoritmo de Dijkstra calcula la distancia mínima a todos los demás nodos en un grafo con pesos (sin aristas negativas):

- v tiene distancia a sí mismo 0, d[v] = 0. Los demás nodos tienen distancia inicial ∞.
- En cada paso elegimos el nodo x que está a menor distancia de v (entre los que no procesamos).
- Para cada vecino y de x, actualizamos su distancia, si es que el camino que pasa por x justo antes de ir a y tenga menor costo que d[y].

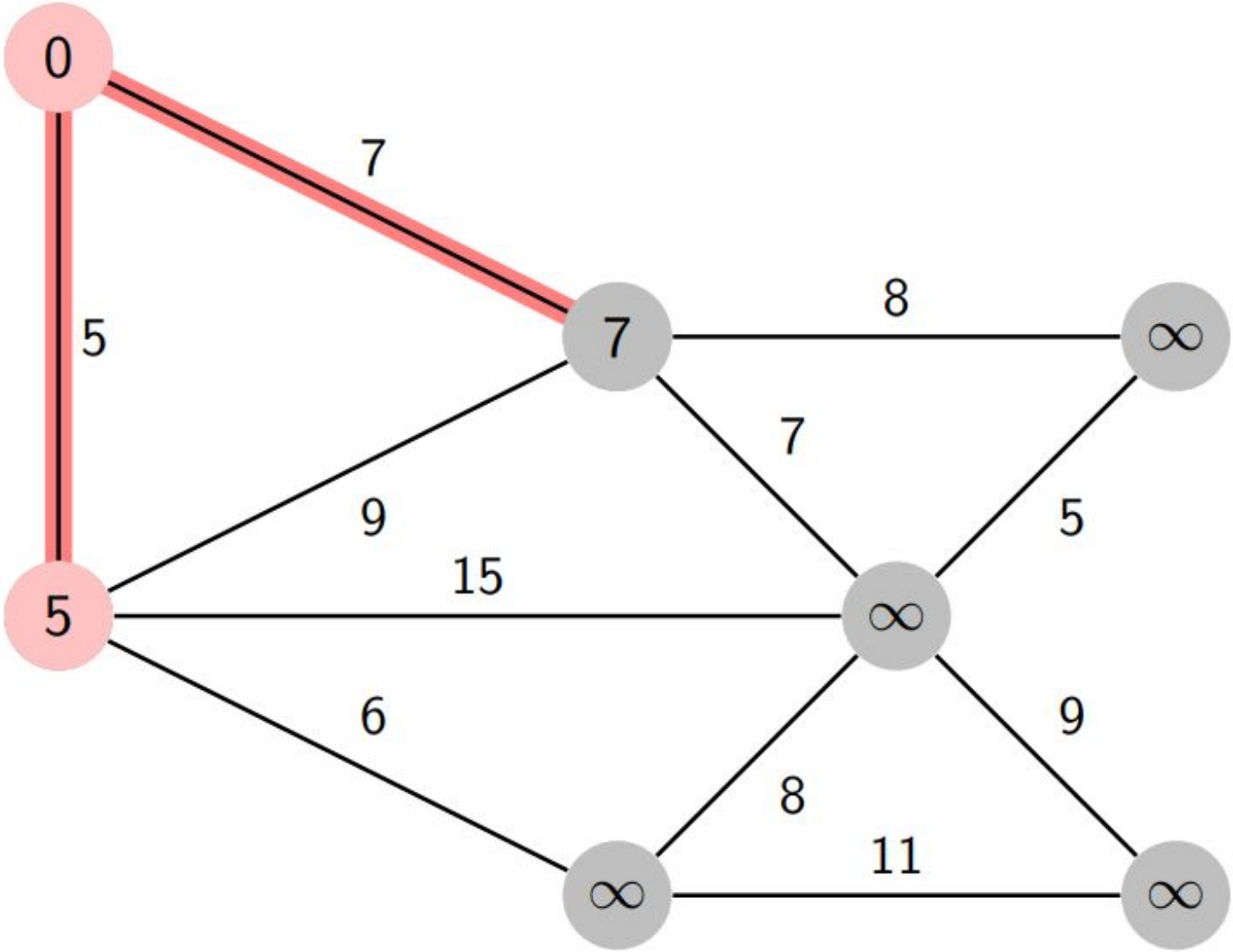
Dijkstra



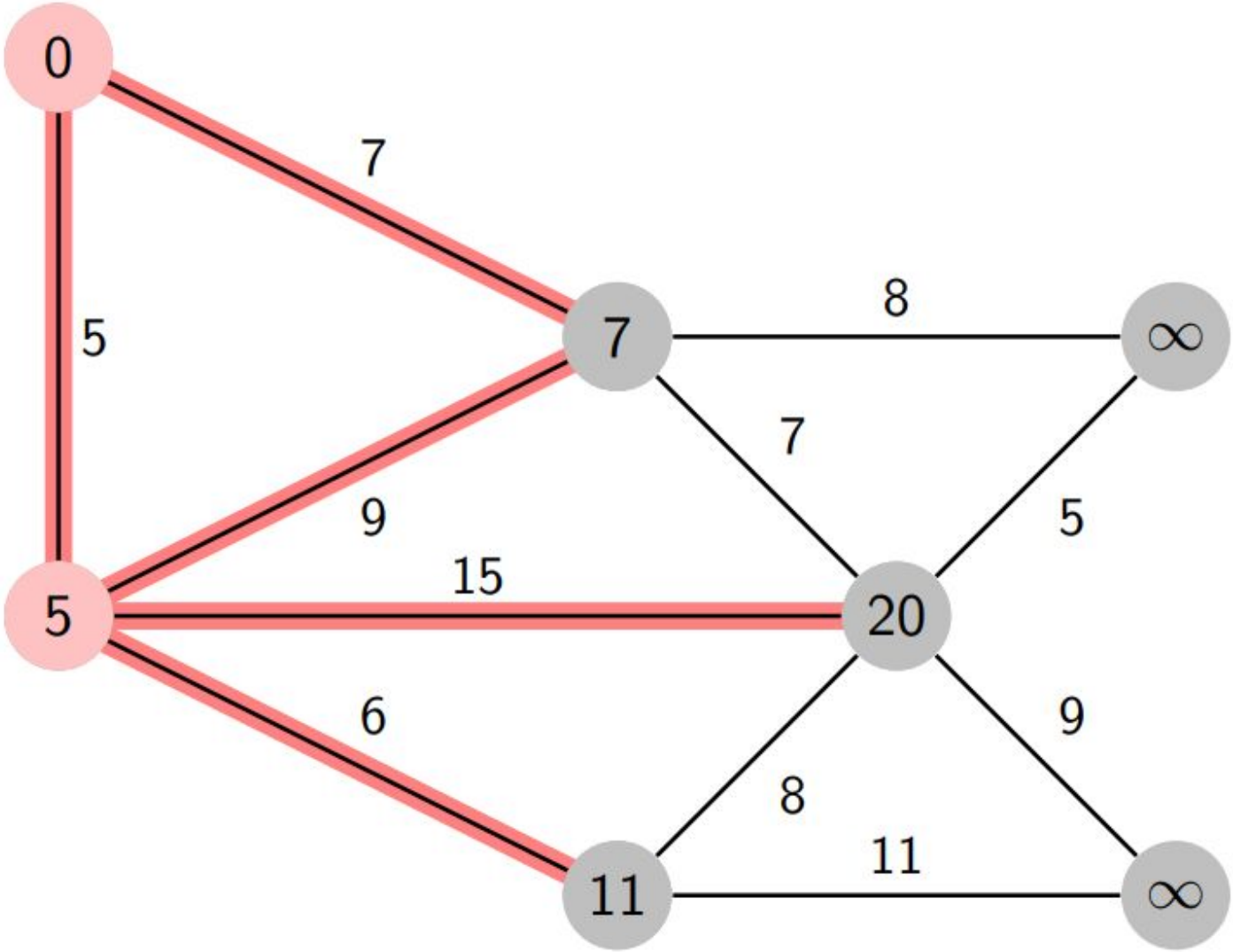
Dijkstra



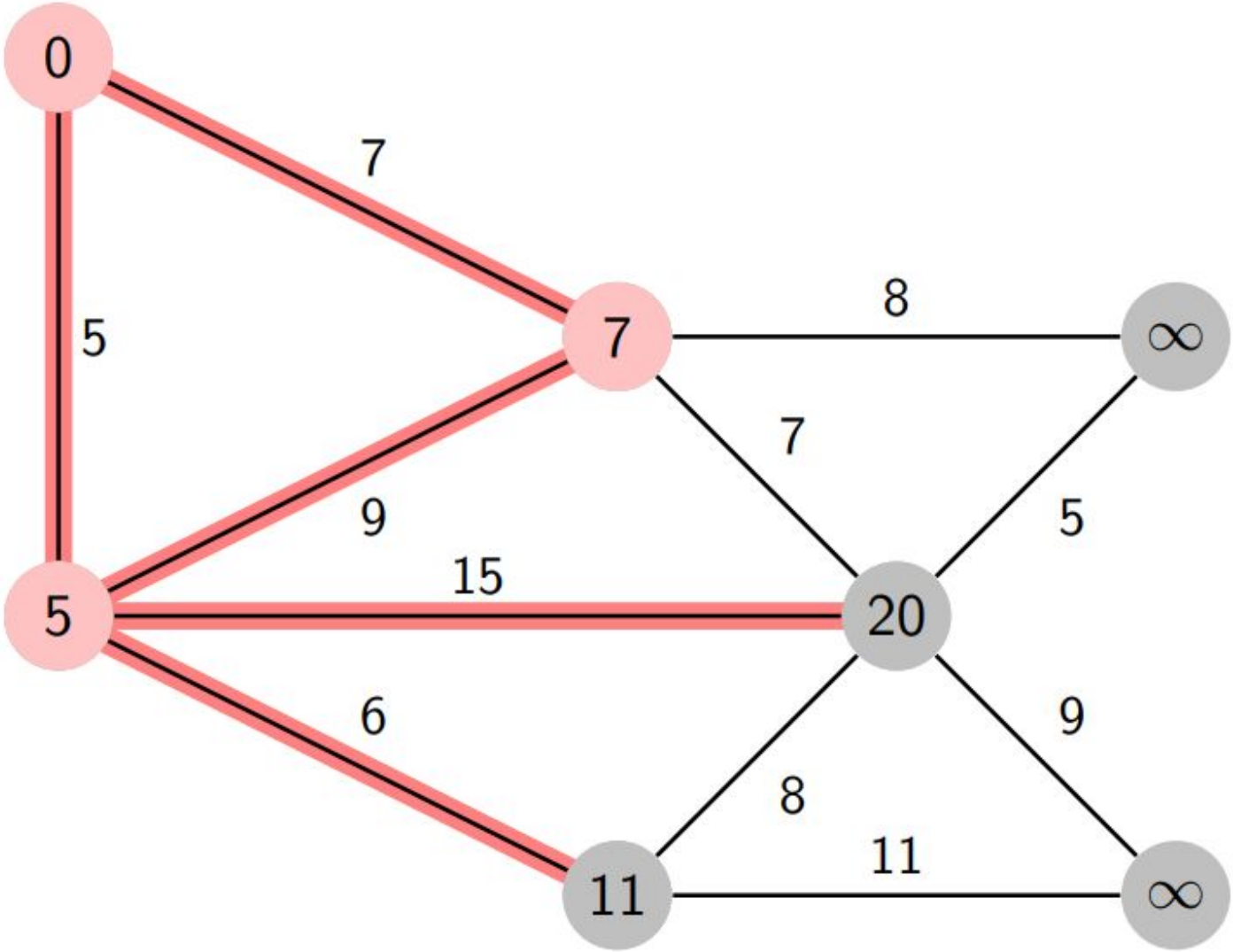
Dijkstra



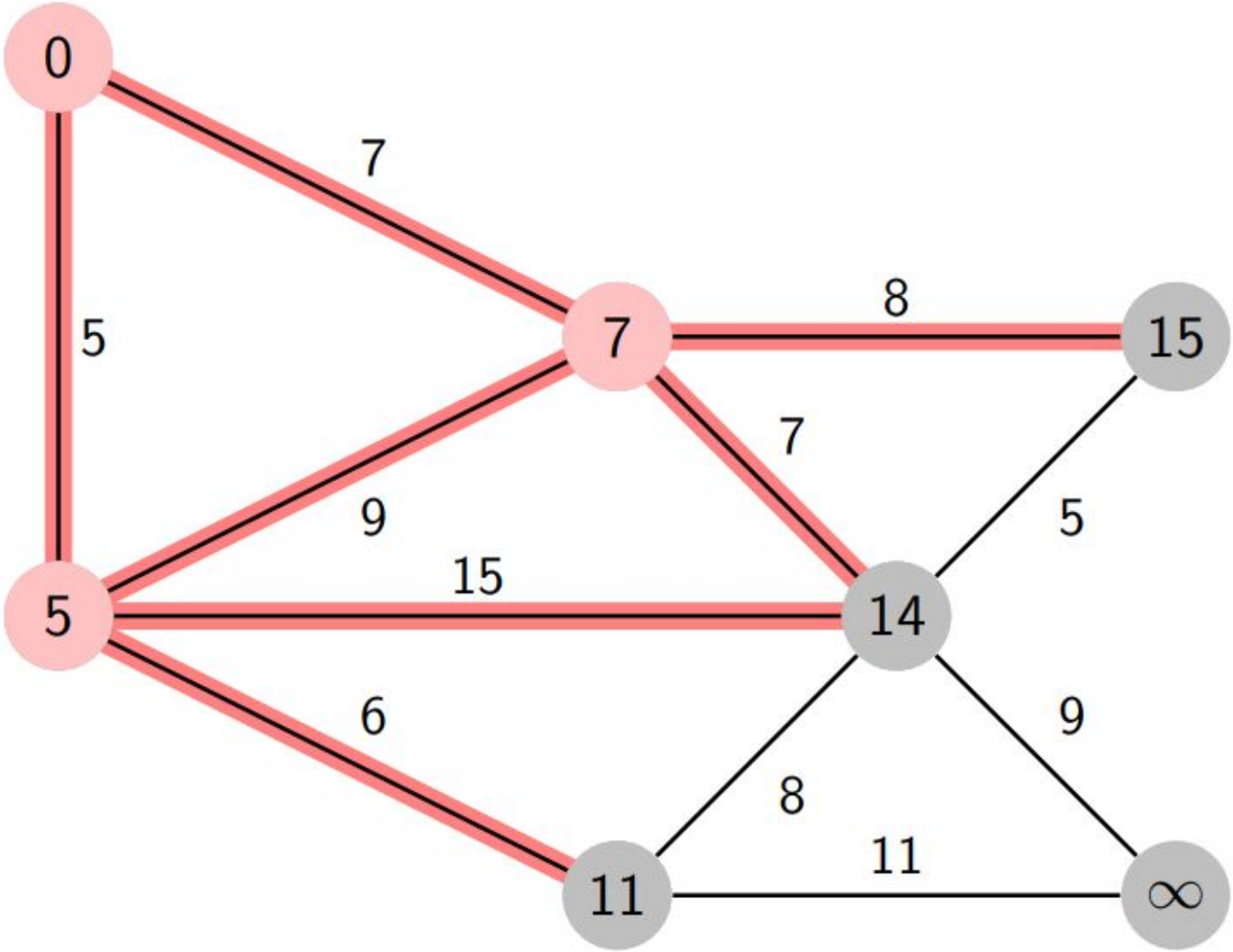
Dijkstra



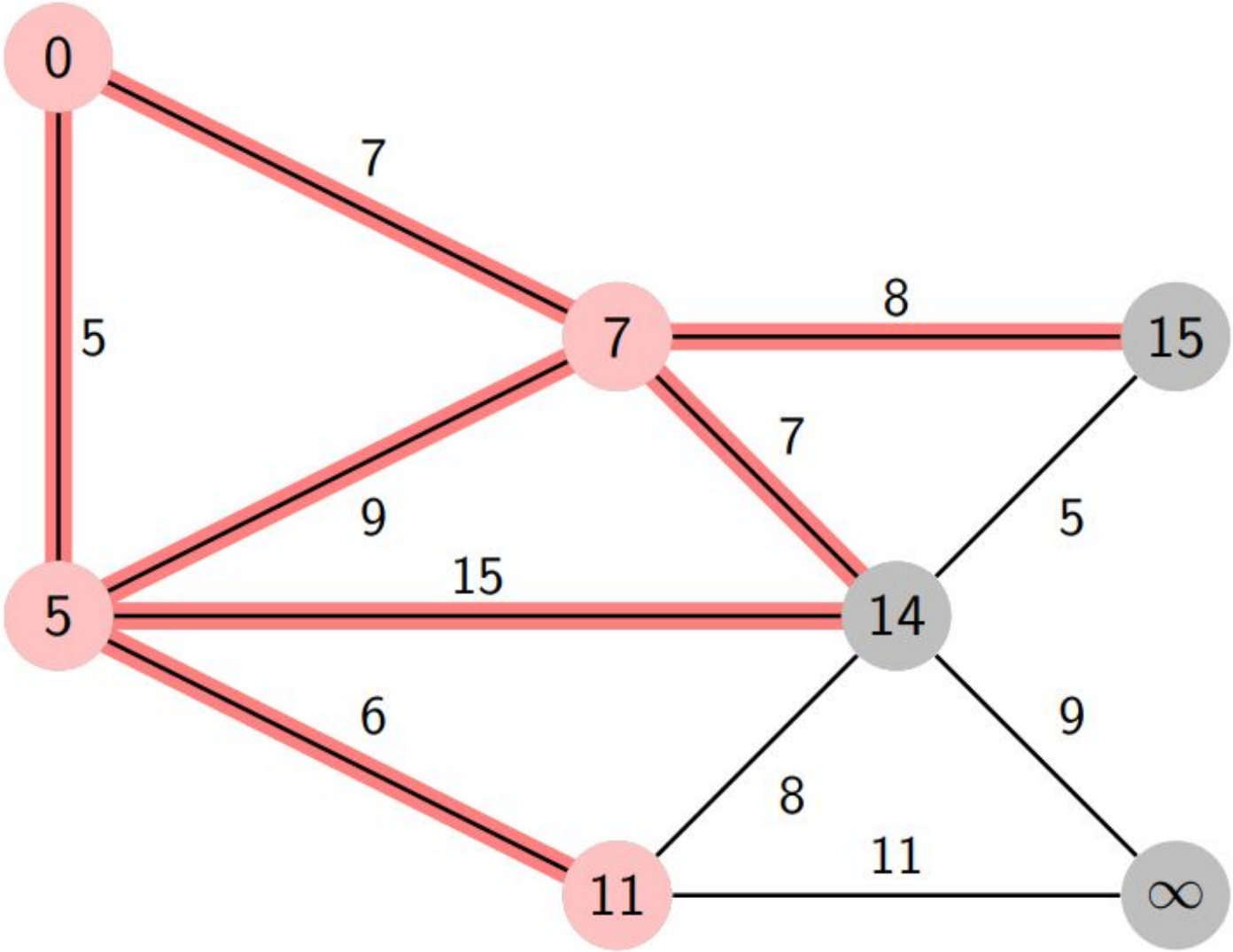
Dijkstra



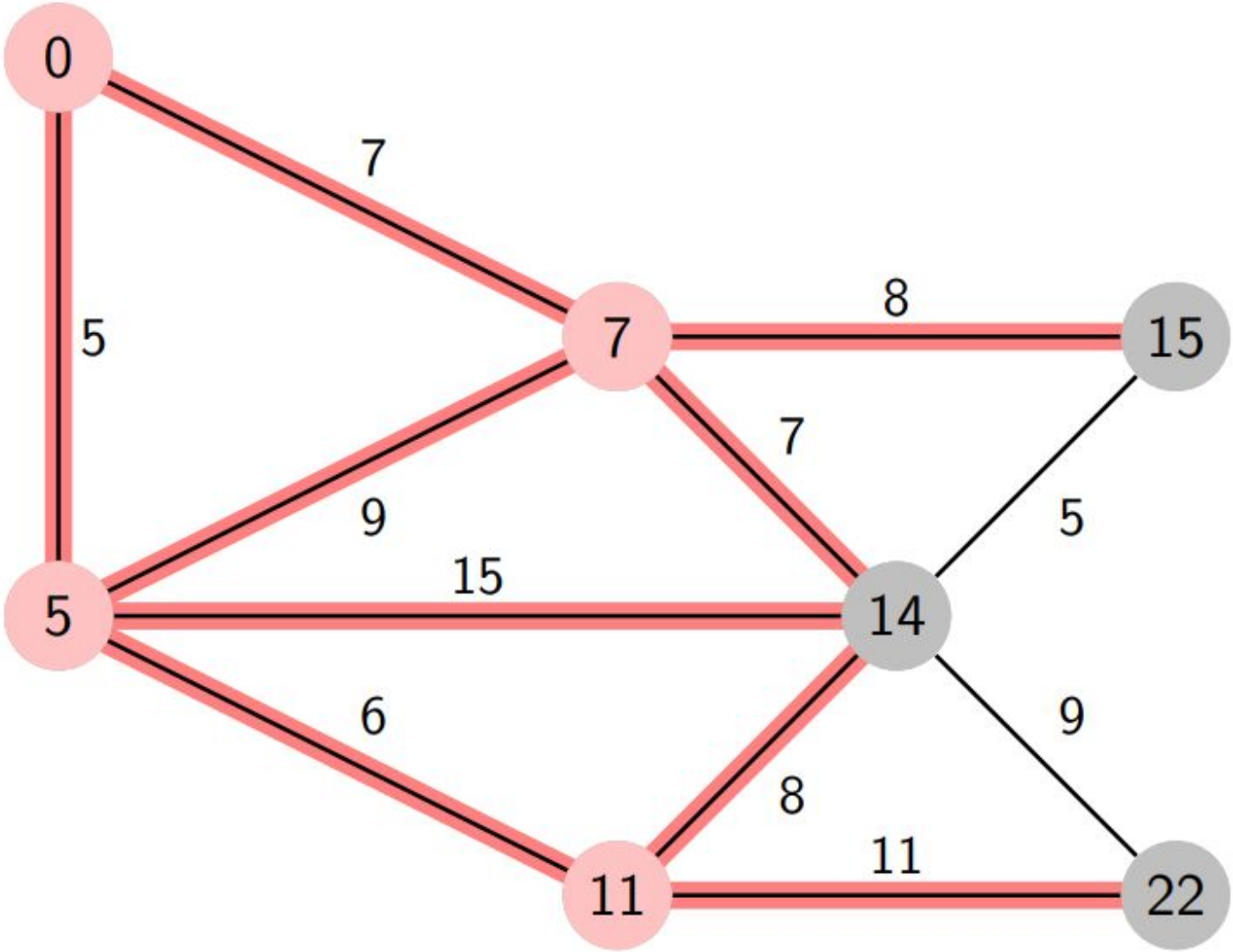
Dijkstra



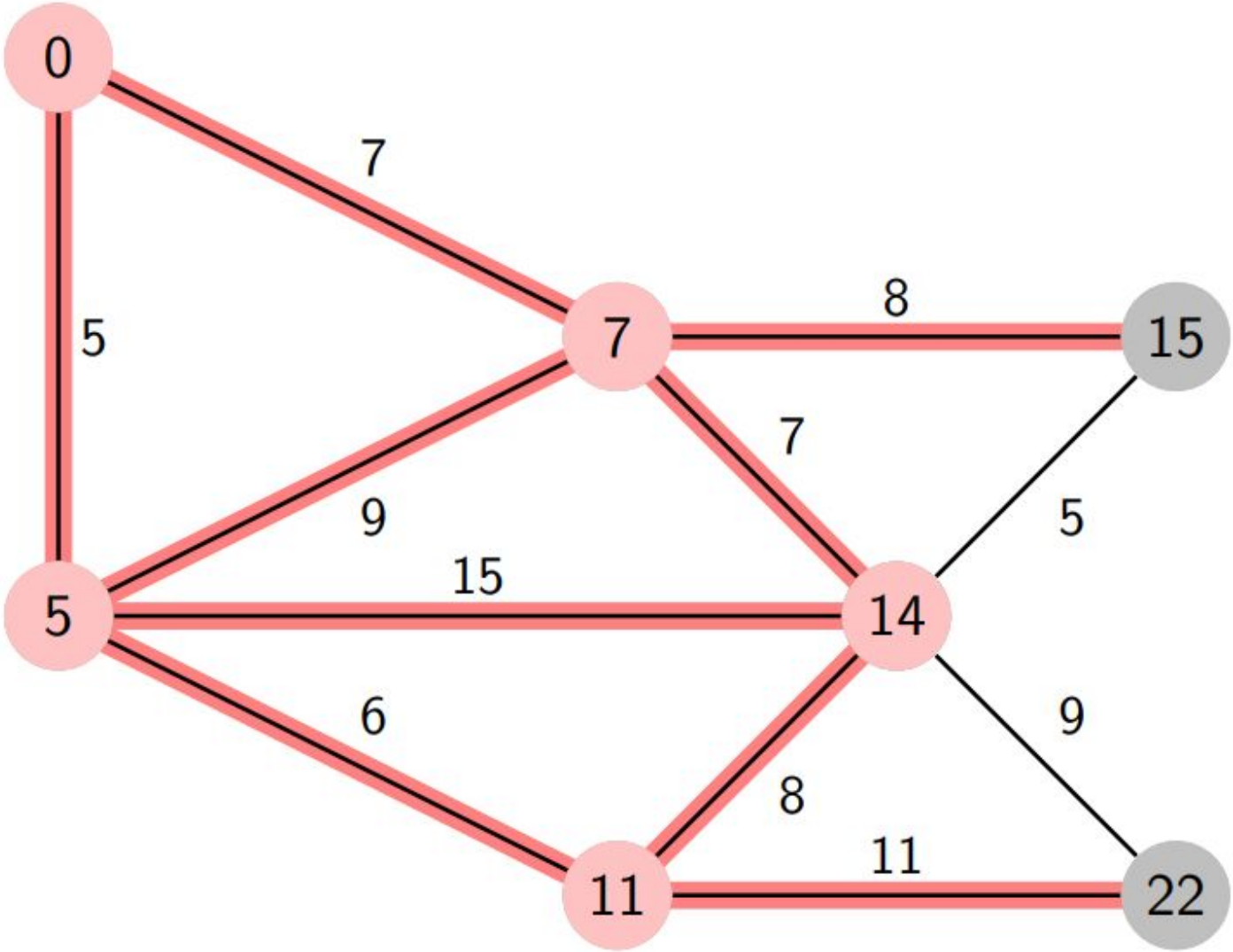
Dijkstra



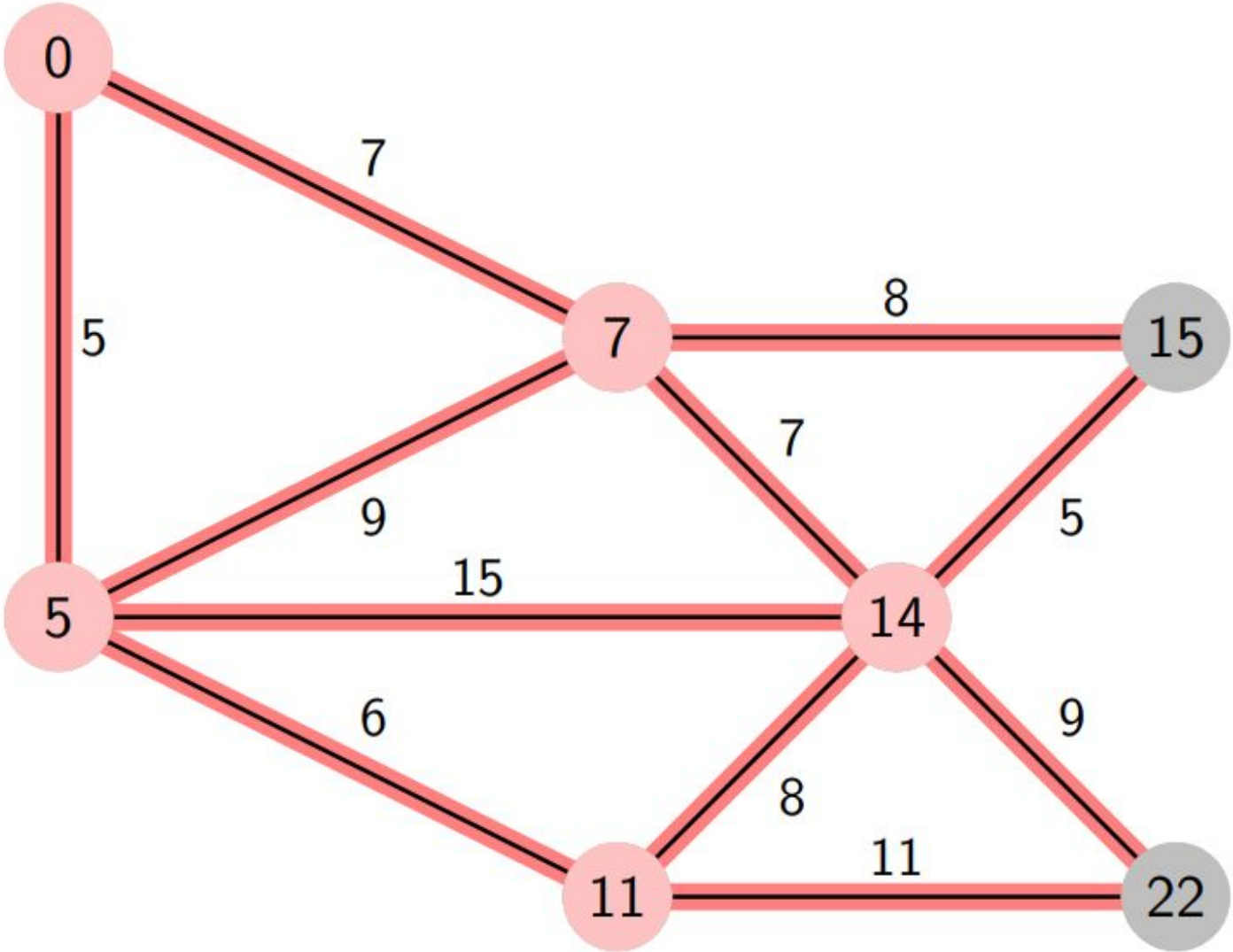
Dijkstra



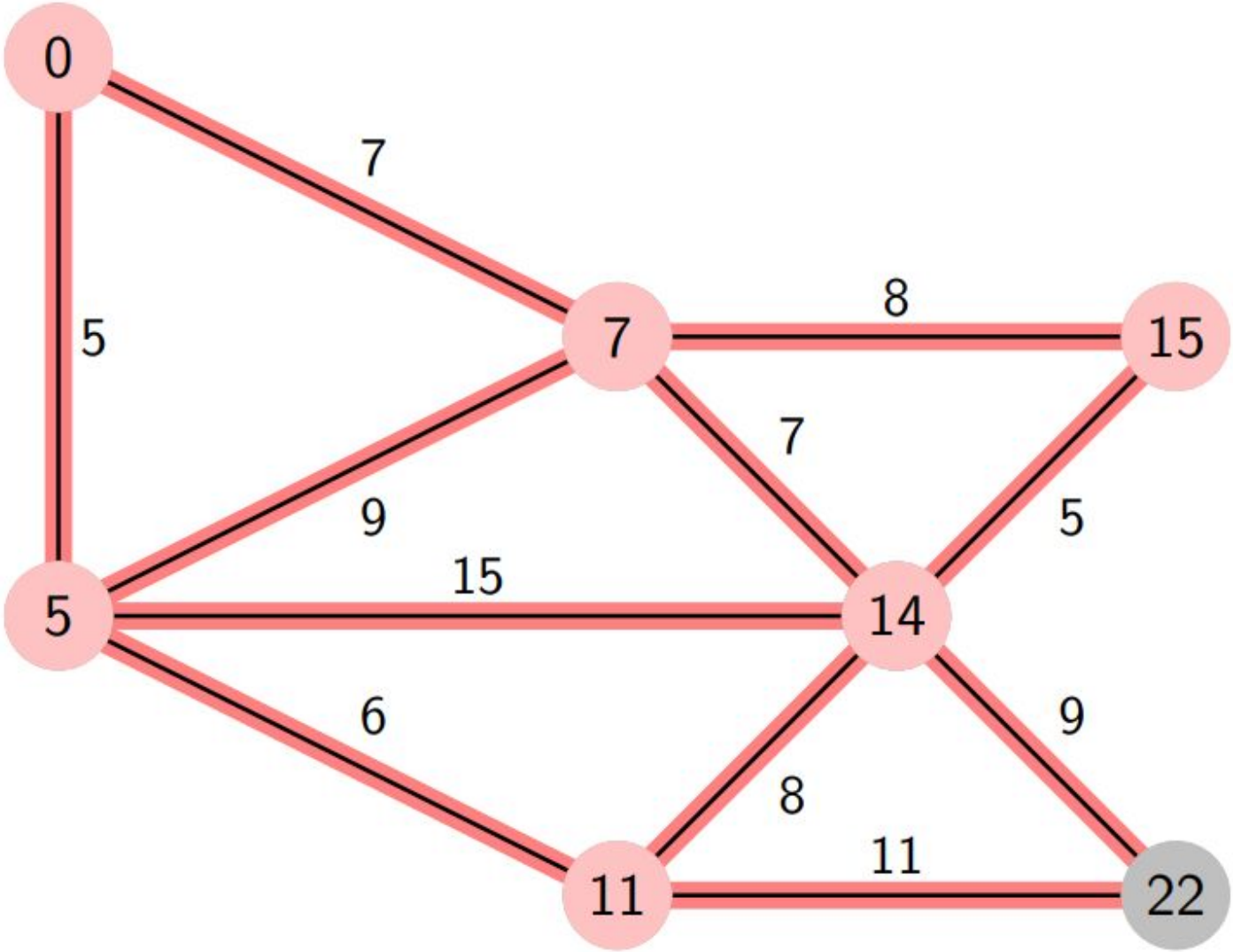
Dijkstra



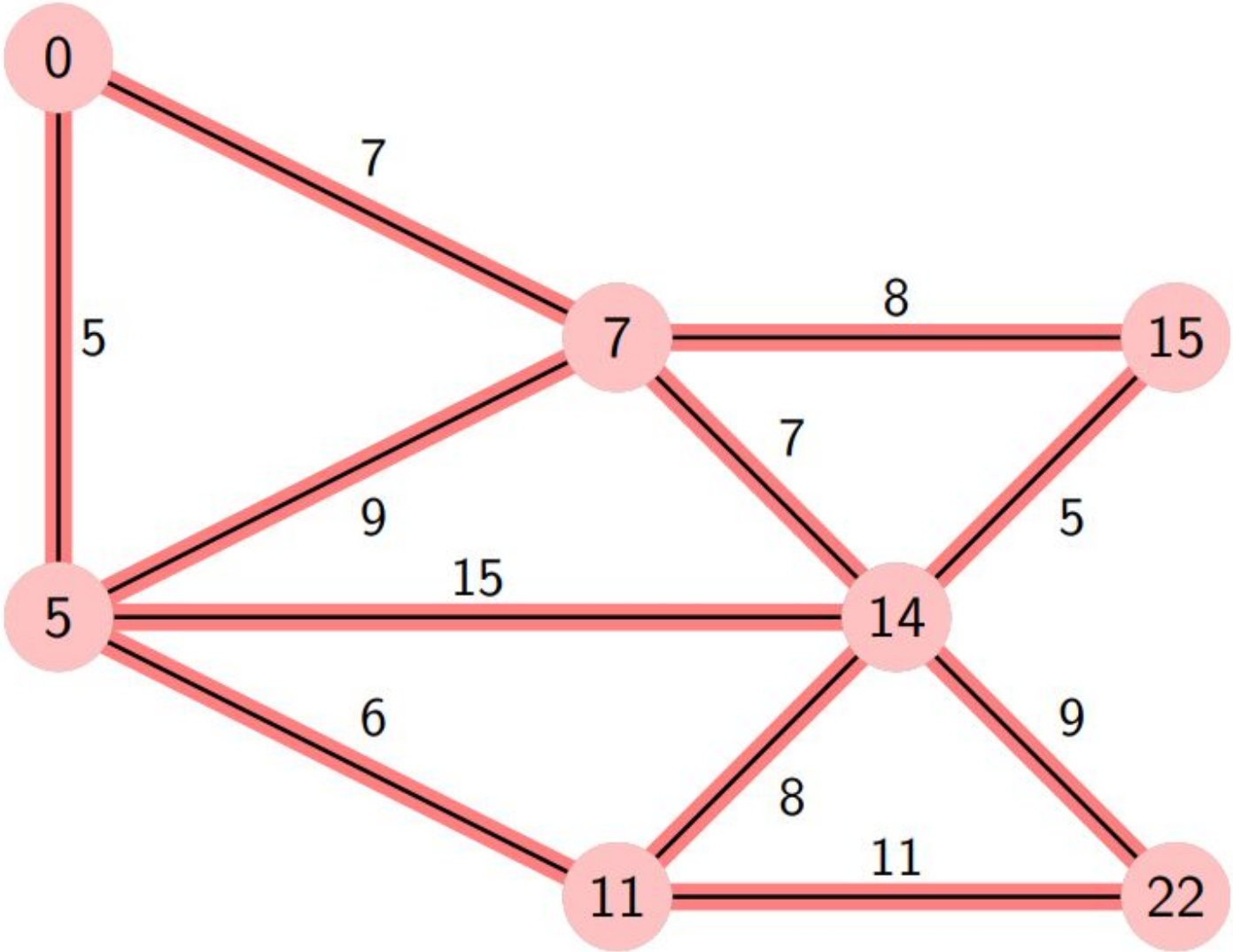
Dijkstra



Dijkstra



Dijkstra



Dijkstra

Se puede implementar en O(|E|log(|E|)), usando una cola de prioridad para elegir el nodo a menor distancia (suele ser necesario usar esta versi´on en las competencias).

Dijkstra no funciona si algunas aristas tienen pesos negativos, cuando hay pesos negativos, puede que la distancia entre algún par de nodos sea $-\infty$, en caso de que exista algún ciclo negativo.

Dijkstra

```
vector<int> dijkstra(vector<vector<pair<int, int>>> g, int source)
{
    int n = g.size();
    vector<int> dis(n, 1e9);
    vector<bool> vis(n);
    priority_queue<pair<int, int>> que;
    que.push({0, source});
    dis[source] = 0;
    while(!que.empty())
    {
        int node = que.top().s;
        que.pop();
        if(vis[node]) continue;
        vis[node] = 1;
        for(ii ed : g[node])
            if(dis[ed.f] > dis[node] + ed.s)
            {
                dis[ed.f] = dis[node] + ed.s;
                que.push({-dis[ed.f], ed.f});
            }
    }
    return dis;
}
```

Bellman-Ford

Dado un nodo (v), el algoritmo de Bellman-Ford calcula la distancia mínima a todos los demás nodos en un grafo con pesos, funciona con aristas negativas:

- Seteamos d[v] = 0, para los demás nodos seteamos su distancia a ∞.
- Iteramos n-1 veces, en cada iteración intentamos relajar la distancia a los nodos usando todas las aristas. Se puede demostrar que luego de la i-ésima iteración, ya se han encontrado todos los caminos mínimos que usan i o menos aristas.
- Hacemos 1 iteración más, si la distancia a algun nodo disminuye, el grafo contiene un ciclo negativo.

Bellman-Ford

```
struct edge{
    int u, v, cost;
};
vector<int> bellmanFord(int n, int m, vector<edge> &e, int s){
    vector<int> dis(n, 1e9);
    dis[s] = 0;
    for (int i = 0; i< n-1; i++)
        for (int j = 0; j < m; j++)
            dis[e[j].v] = min(dis[e[j].v], dis[e[j].u] + e[j].cost);
    return dis;
}
```

Floyd-Warshall

- Calcula las distancia entre cada par de nodos, lo hace usando programación dinámica.
- Inicialmente, d[x][y] (distancia de x a y) es el peso de la arista de x a y si esta existe, ∞ si no existe, es 0 si x = y.
- Procesamos los nodos uno por uno, cuando procesamos el nodo k, para cada par de nodos i, j actualizamos su distancia como el mínimo entre lo que ya había antes y la distancia si pasamos por k (es decir, d[i][k] + d[k][j]).

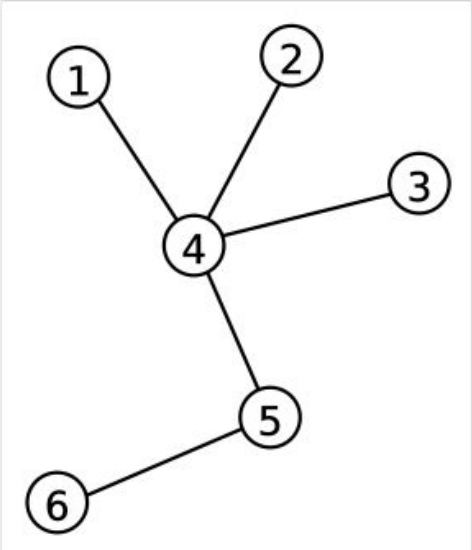
Floyd-Warshall

```
void floyd(vector<vector<int> > &dis){
   for(int k=0;k<n;k++)
      for(int i=0;i<n;i++)
         for(int j=0;j<n;j++)
            dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
}
```

- La complejidad es claramente O(n^3).
- Para pesos negativos, x está en un ciclo negativo, si d[x][x] < 0.
- La distancia de x a y es -∞ si hay un nodo z que está en un ciclo negativo y podemos ir de x a z y de z a y.

Árboles

- Un árbol es un grafo conexo sin ciclos.
- Un árbol cumple que |E| = |V| - 1.



Union-find

Union-find es una estructura de datos que mantiene conjuntos disjuntos.

- **find(x)**: Devuelve un id del conjunto en el que está el elemento x. Se puede usar para ver si dos nodos están el mismo conjunto chequeando find(x) == find(y).
- **union(x, y)**: Une los conjuntos a los que pertenecen x e y. Es decir, junta las componentes de x y de y en la misma componente.

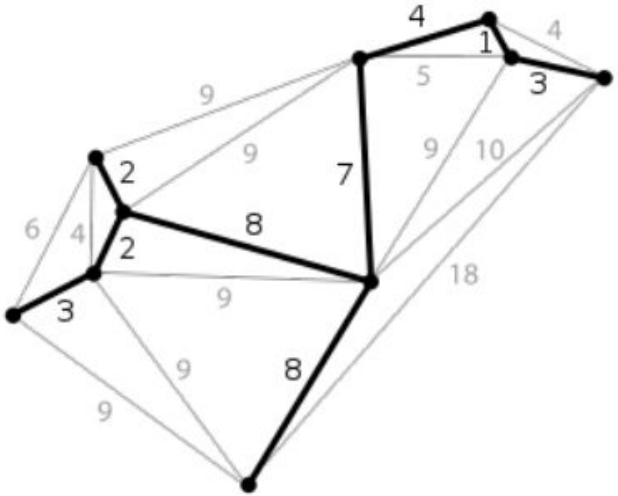
Ambas operaciones se pueden realizar de manera muy eficiente (prácticamente O(1))

Union-find

```
struct unionFind {
  vi p;
  unionFind(int n) : p(n, -1) {}
  int findParent(int v) {
    if (p[v] == -1) return v;
    return p[v] = findParent(p[v]);
  }
  bool join(int a, int b) {
    a = findParent(a);
    b = findParent(b);
    if (a == b) return false;
    p[a] = b;
    return true;
  }
};
```

Kruskal (árbol de Expansión Mínimo (MST))

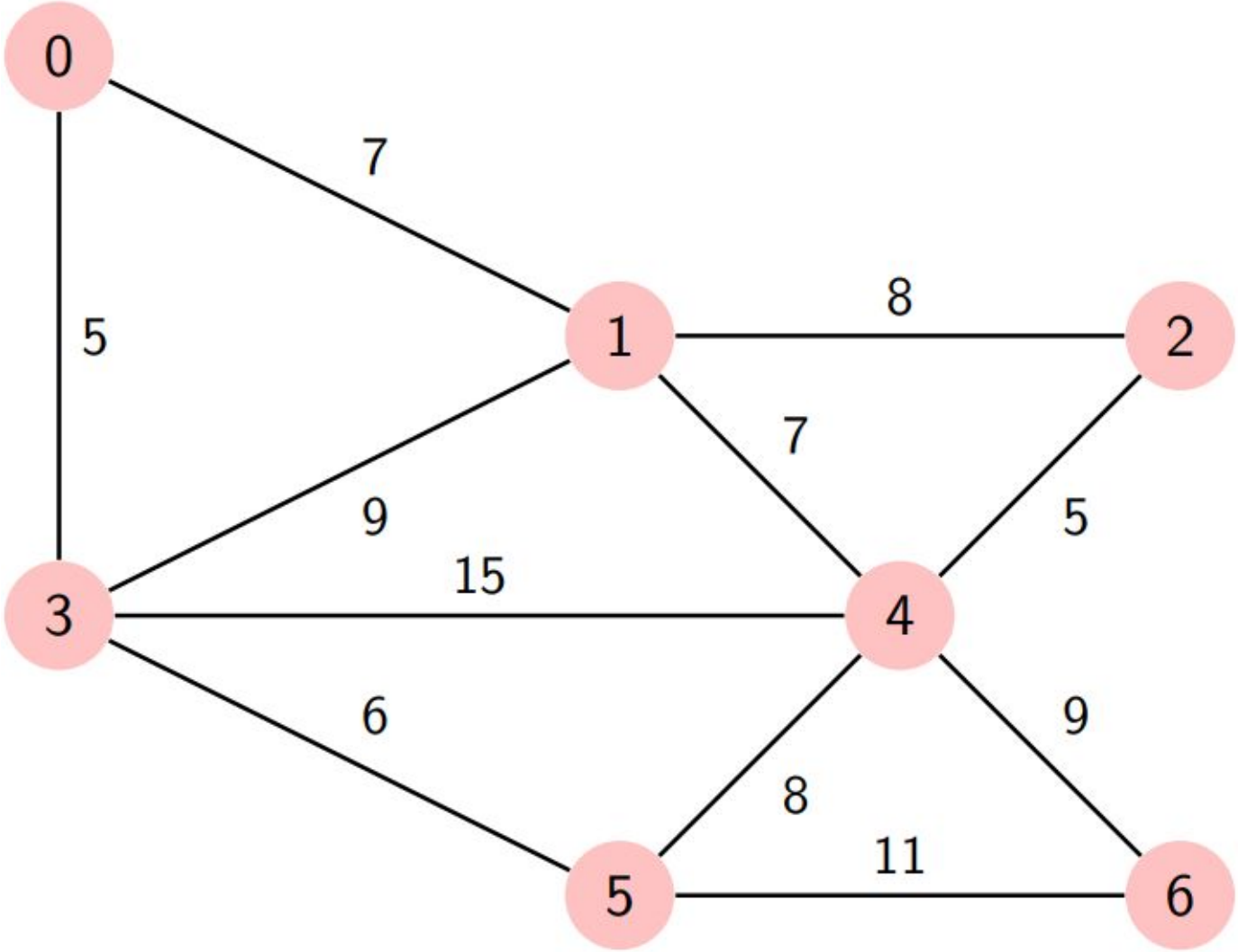
Dado un grafo conexo G, un árbol de expansión de G es un árbol que tiene todos los nodos de G y cuyas aristas también son aristas de G. Si las aristas de G tienen pesos, un "árbol de expansión mínimo" de G es uno que cumple que la suma de sus aristas es la menor posible.



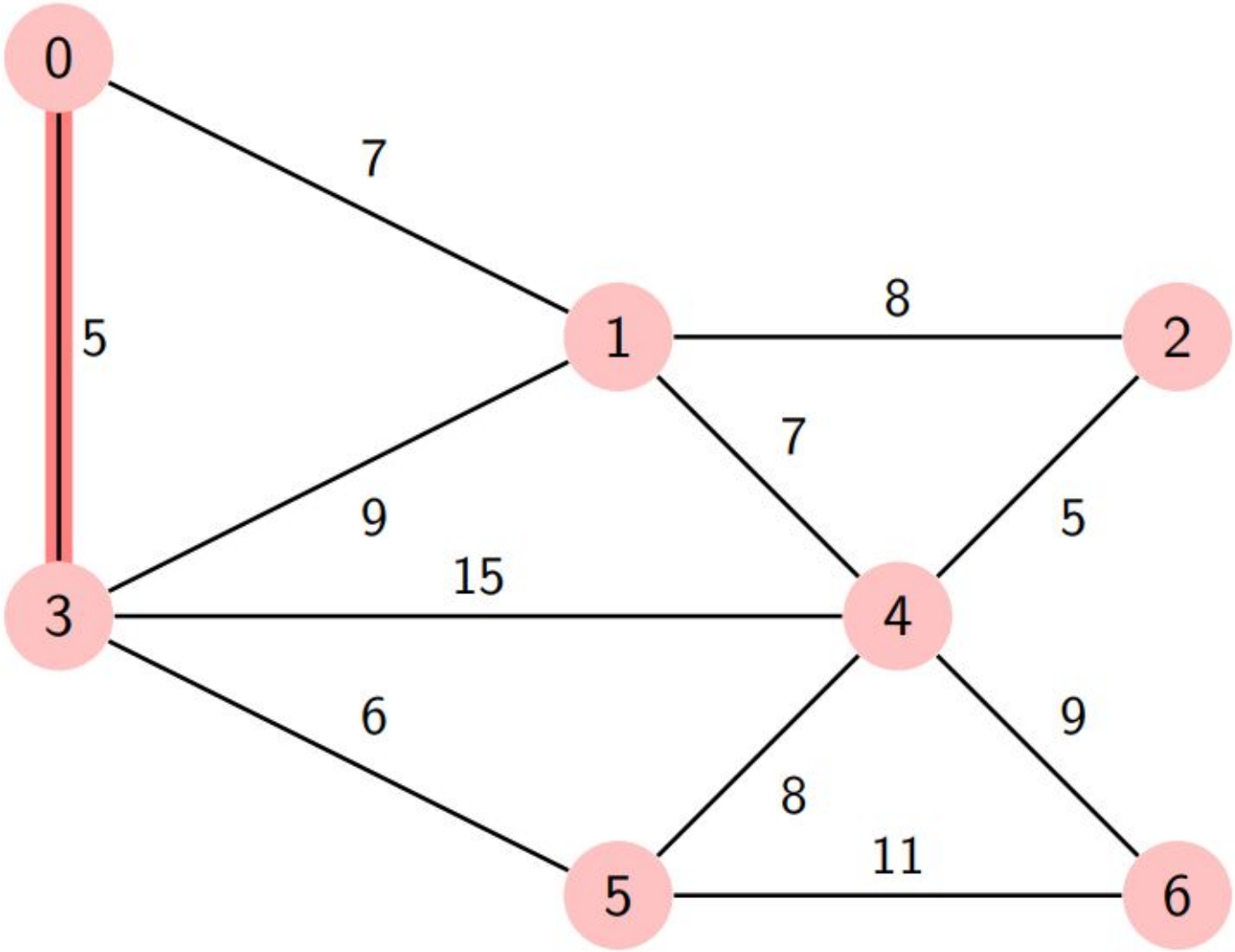
Kruskal (Algoritmo)

- Mantener un union-find con las componentes conexas determinadas por aristas que se agregan al árbol.
- Ordenar las aristas por peso de menor a mayor.
- Recorrer cada arista: Si los nodos que conecta están en distintas componentes conexas, agregamos la arista al árbol y hacemos unión de los dos nodos en el union-find.
- La complejidad es $O(|E|\log(|N|))$ por que se ordenan las aristas
- Para Kruskal, no hace falta representar el grafo como lista de adyacencia, sino simplemente como un arreglo con las aristas.

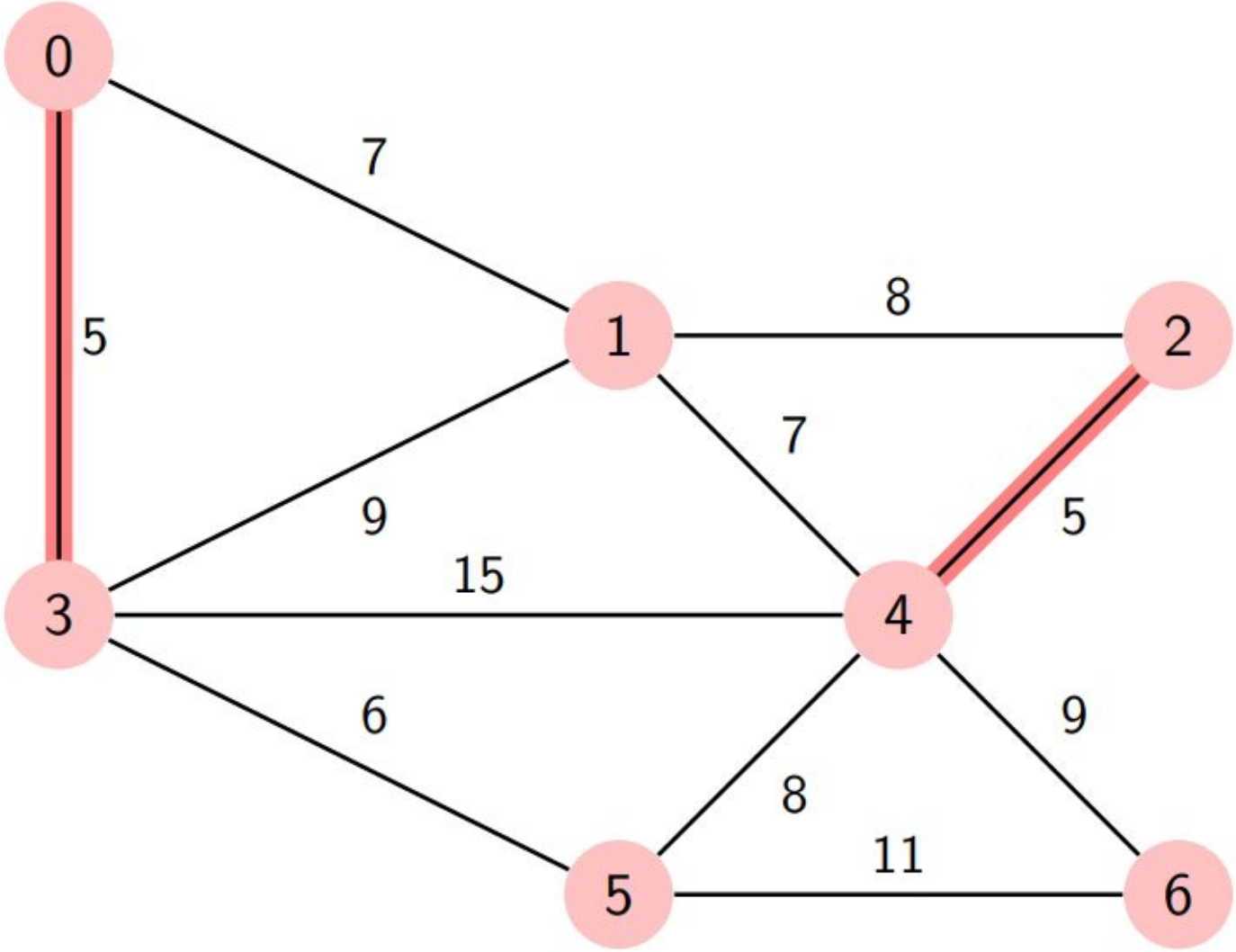
Kruskal



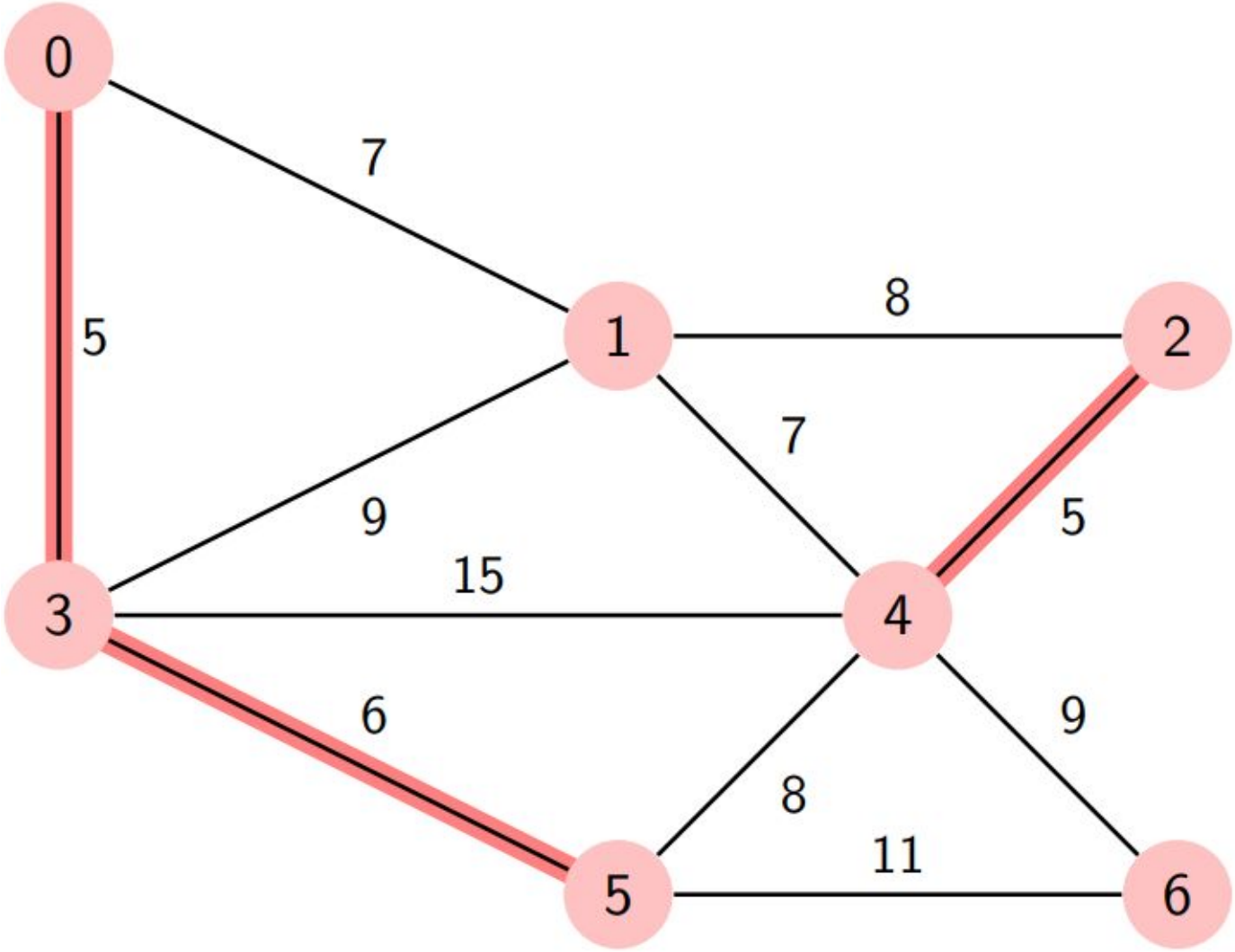
Kruskal



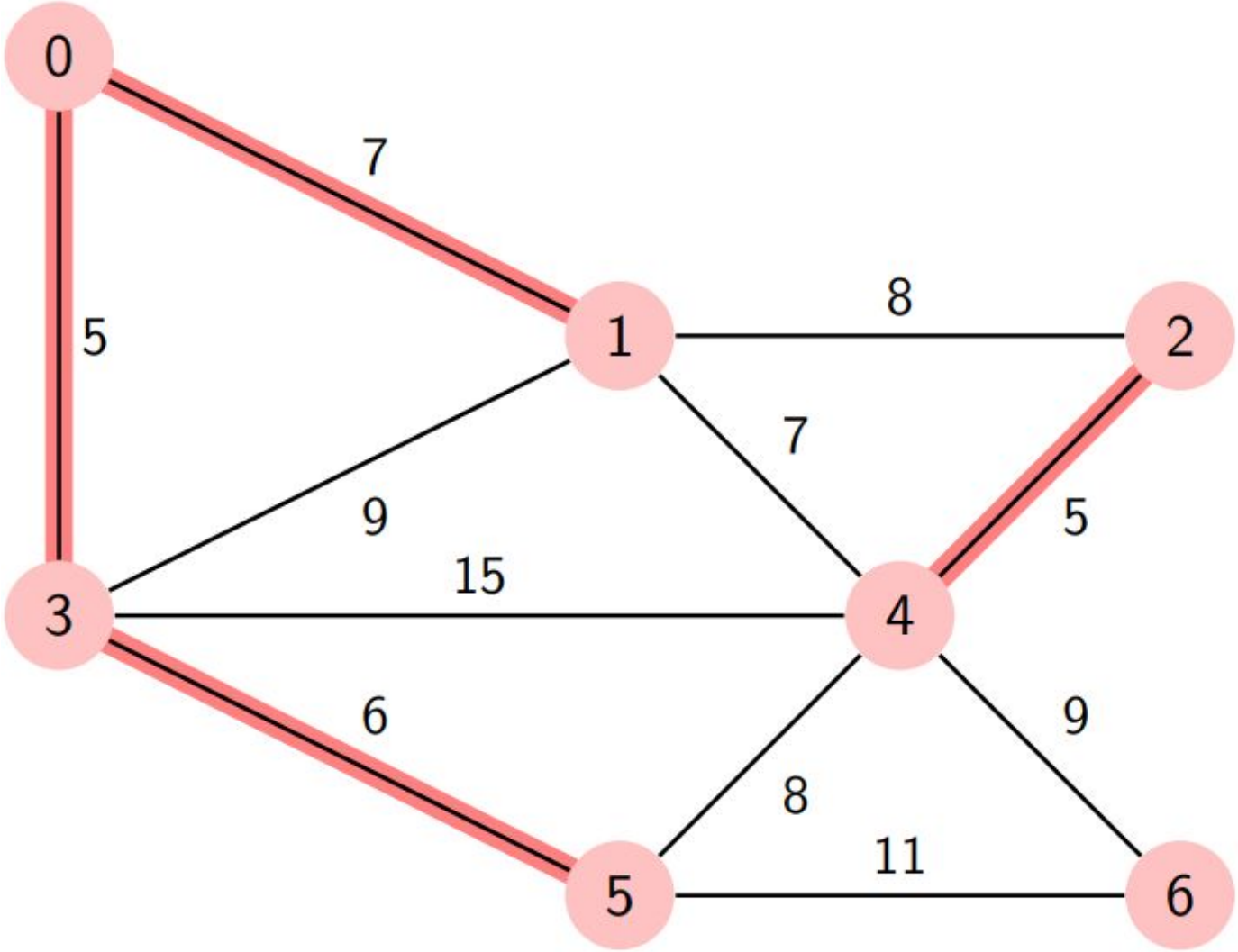
Kruskal



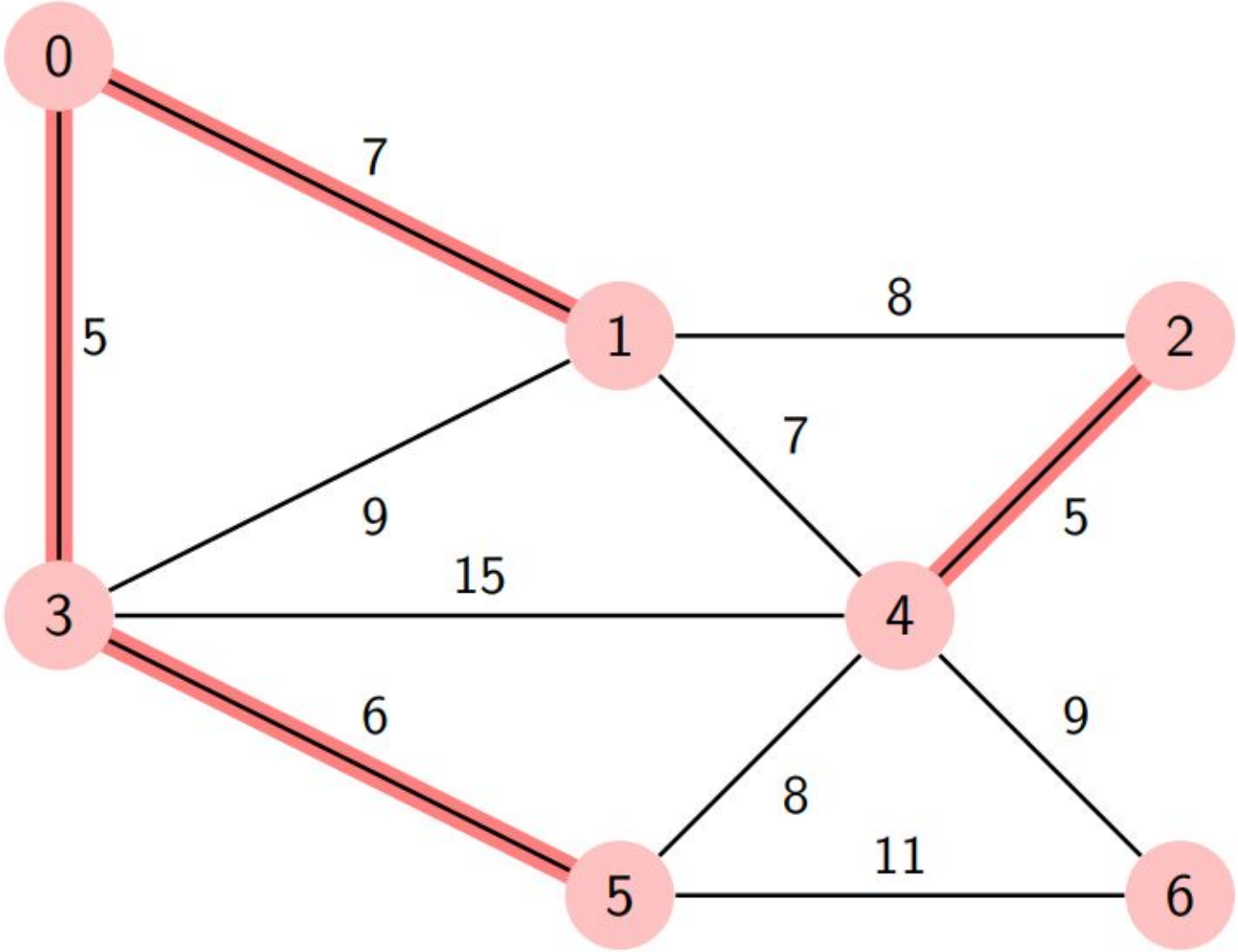
Kruskal



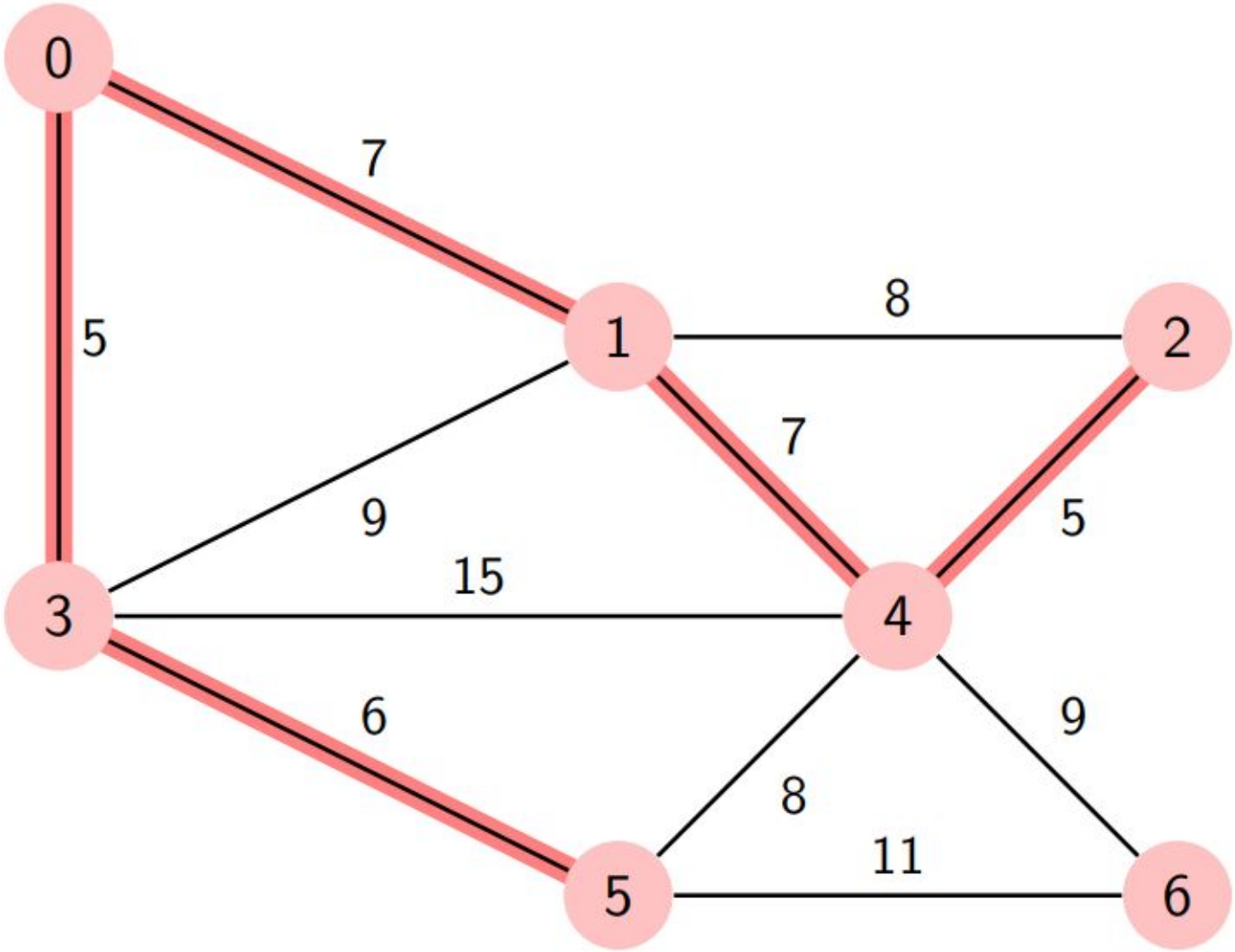
Kruskal



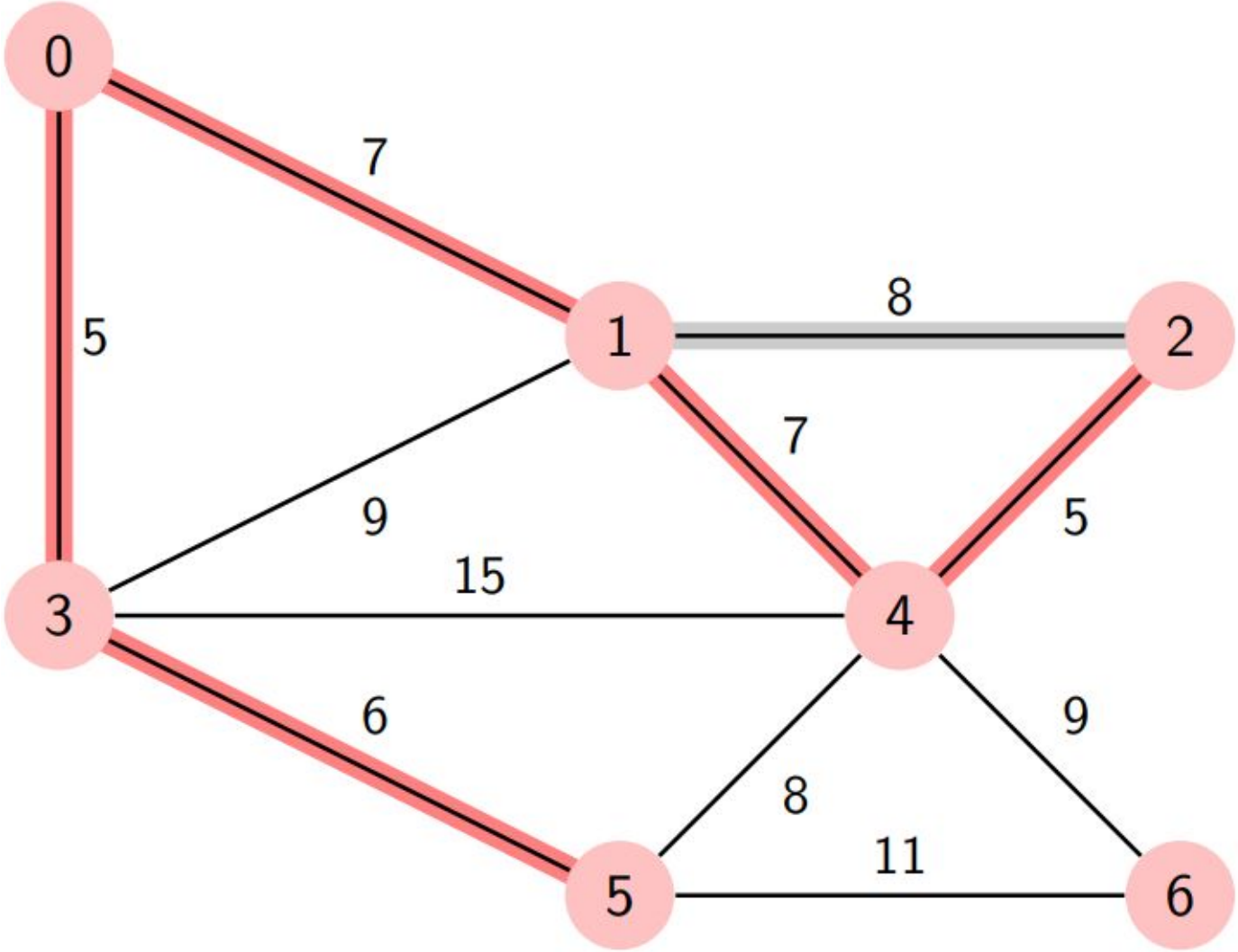
Kruskal



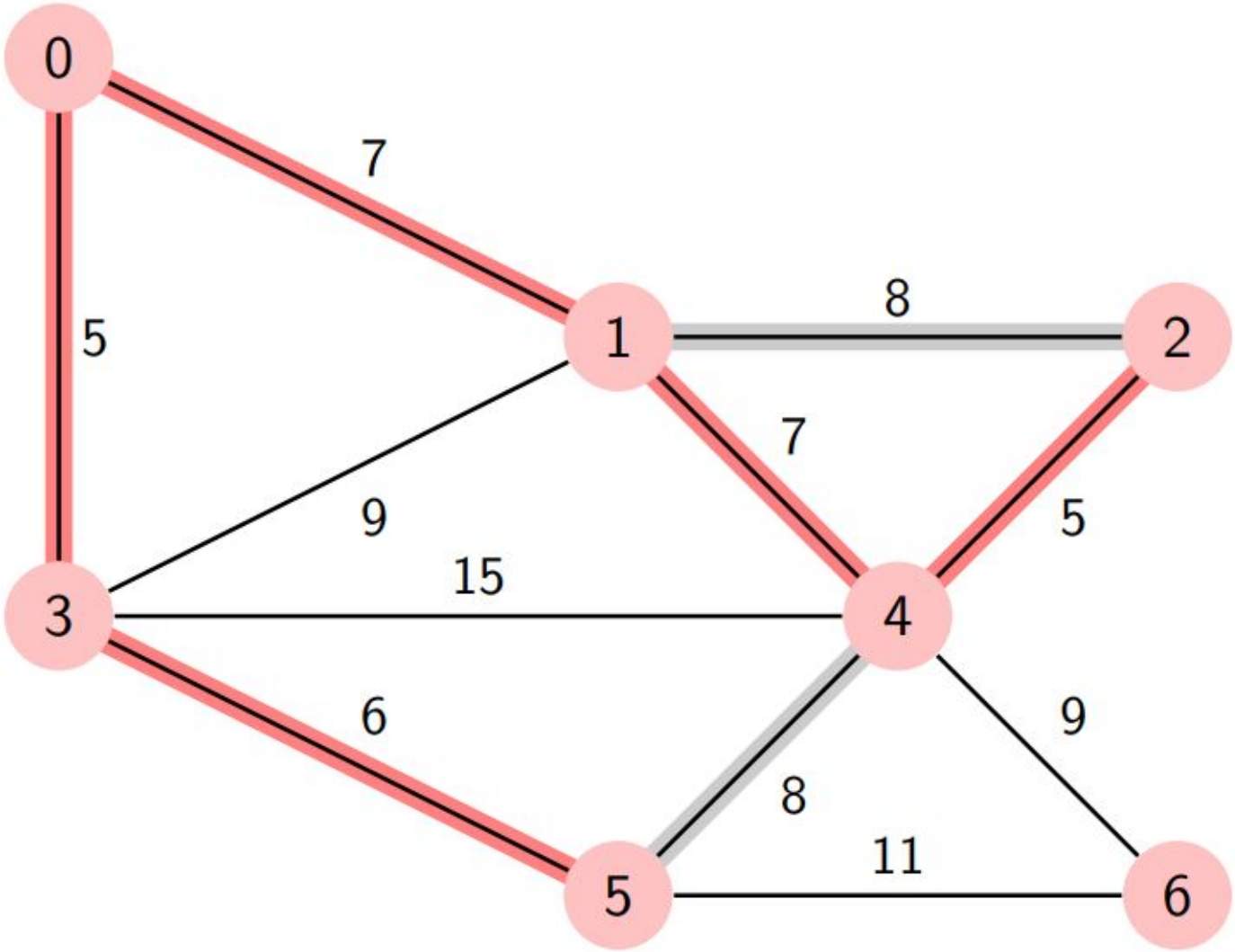
Kruskal



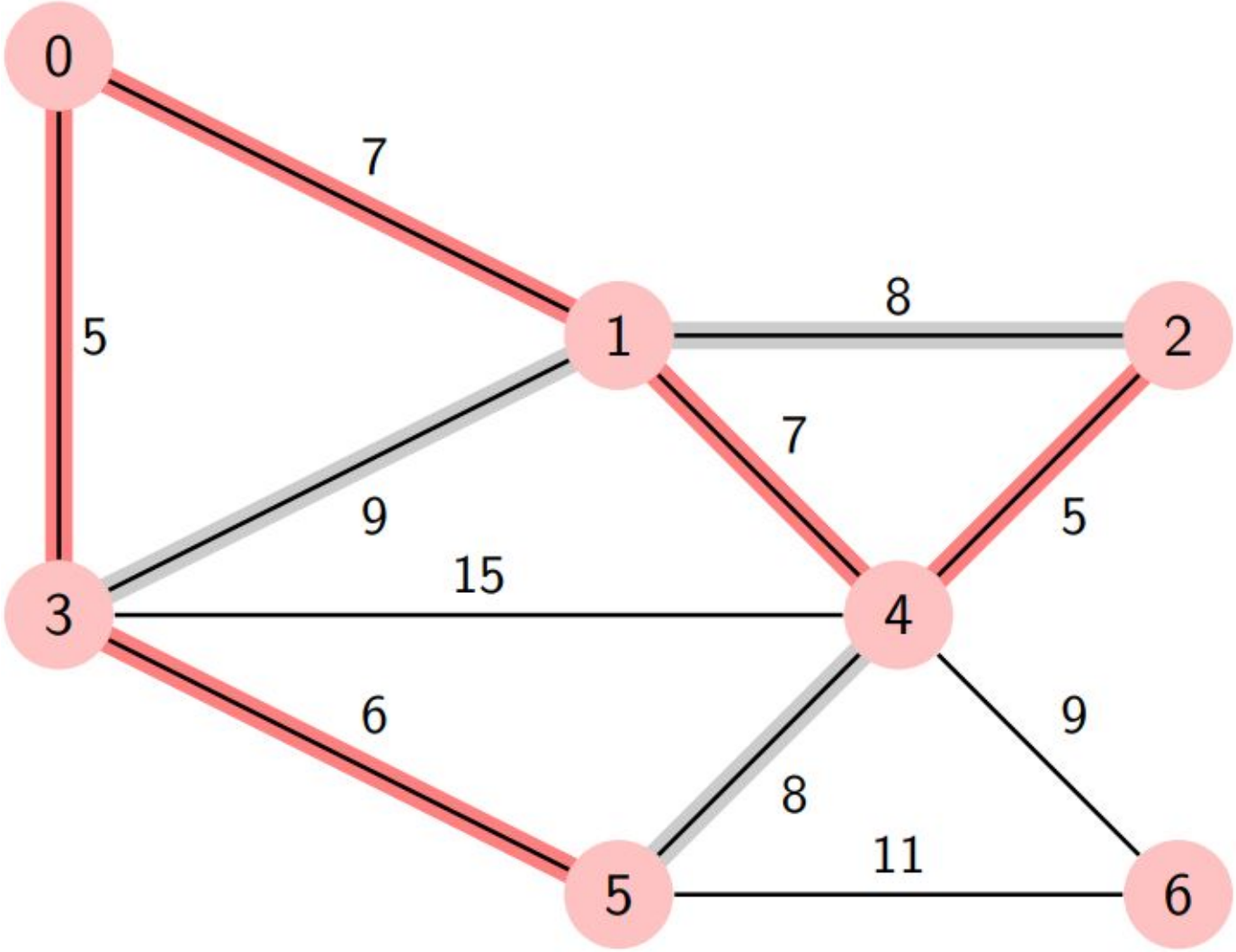
Kruskal



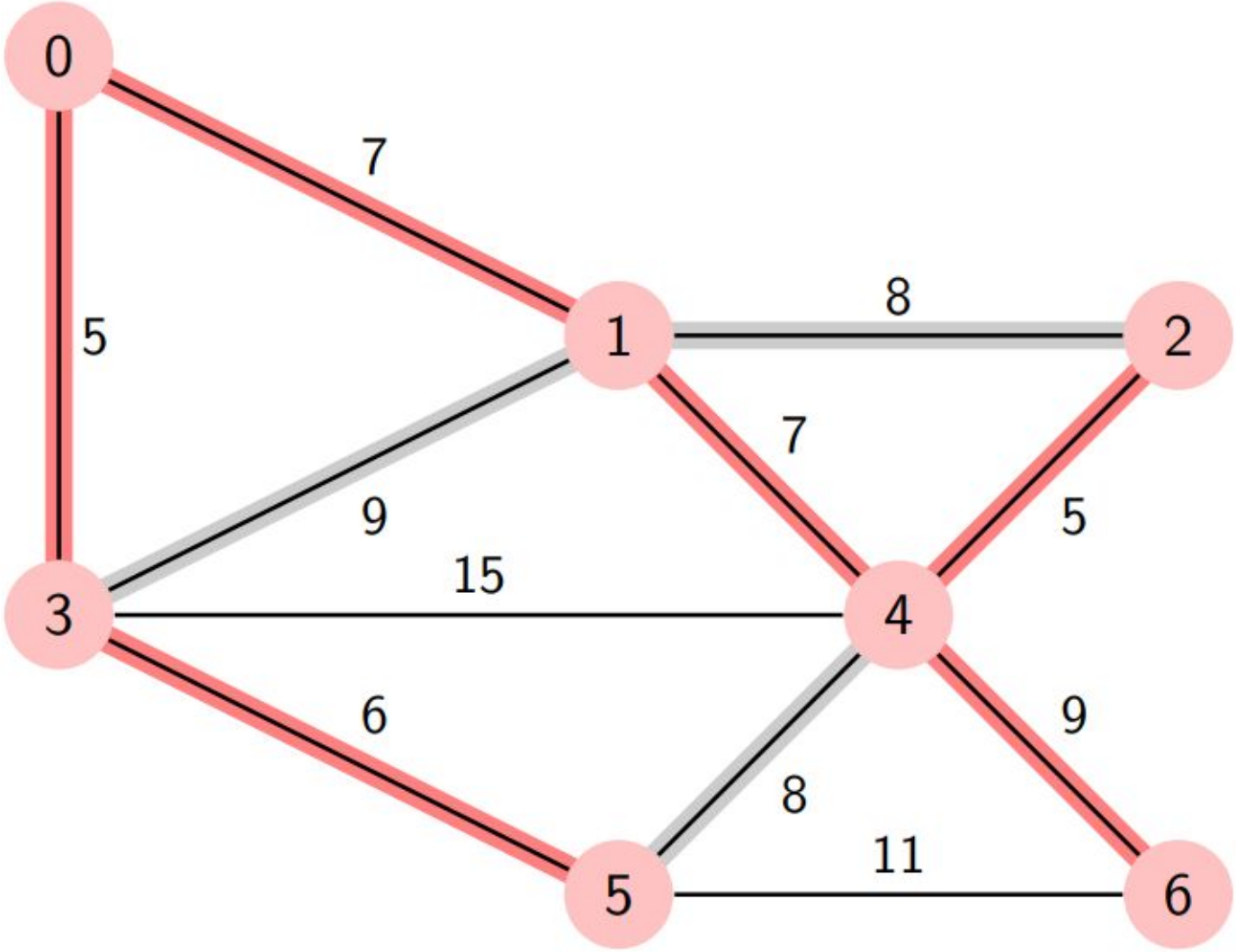
Kruskal



Kruskal



Kruskal



Kruskal

```
int kruskal(vector<pair<int, pair<int, int>>> edges, int n) {
//edges: lista de aristas en la forma {peso, {nodo1, nodo2}}
    sort(edges.begin(),edges.end()); // ordena por peso
    unionFind uni(n);
    int cont = 0, res = 0;
    for(auto e: edges){
        int costo = e.first, u = e.second.first, v = e.second.second;
        if(uni.join(u, v))
        {
            cont++;
            res += costo;
        }
        if(cont == n - 1) // completamos el árbol?
        return res;
    }
    return -1; // si llegamos hasta acá entonces no es conexo
     }
```

Árbol De Expansión Mínimo: Propiedades

- Un MST de un grafo es único si el peso de todas las aristas es distinto. Caso contrario, puede haber varios árboles de expansión mínima (los algoritmos específicos suelen dar como resultado uno de los posibles árboles de expansión mínima).
- El árbol de expansión mínima es también el árbol con el producto mínimo de los pesos de las aristas. (Se puede demostrar fácilmente sustituyendo los pesos de todas las aristas por sus logaritmos).
- En un árbol de expansión mínima de un grafo, el peso máximo de una arista es el mínimo posible de entre todos los árboles de expansión posibles de ese grafo. (Esto se deduce de la validez del algoritmo de Kruskal).
- El árbol de expansión máxima (árbol de expansión en el que la suma de los pesos de las aristas es máxima) de un grafo puede obtenerse de forma similar al MST, cambiando los signos de los pesos de todas las aristas por su opuesto y aplicando a continuación cualquiera de los algoritmos del MTS.